

LIGHTMEM: LIGHTWEIGHT AND EFFICIENT MEMORY-AUGMENTED GENERATION

Jizhan Fang[♣], Xinle Deng[♣], Haoming Xu[♣], Ziyang Jiang[♣], Yuqi Tang[♣], Ziyen Xu[♣],
Shumin Deng[◇], Yunzhi Yao[♣], Mengru Wang[♣], Shuofei Qiao[♣], Huajun Chen[♣], Ningyu Zhang^{♣♣*}
[♣]Zhejiang University [◇]National University of Singapore
[♣]State Key Lab. for Novel Software Technology, Nanjing University, P.R. China
{fangjizhan, zhangningyu}@zju.edu.cn

ABSTRACT

Despite their remarkable capabilities, Large Language Models (LLMs) struggle to effectively leverage historical interaction information in dynamic and complex environments. Memory systems enable LLMs to move beyond stateless interactions by introducing persistent information storage, retrieval, and utilization mechanisms. However, existing memory systems often introduce substantial time and computational overhead. To this end, we introduce a new memory system called **LightMem**, which strikes a balance between the performance and efficiency of memory systems. Inspired by the Atkinson–Shiffrin model of human memory, **LightMem** organizes memory into three complementary stages. First, cognition-inspired sensory memory rapidly filters irrelevant information through lightweight compression and groups information according to their topics. Next, topic-aware short-term memory consolidates these topic-based groups, organizing and summarizing content for more structured access. Finally, long-term memory with sleep-time update employs an offline procedure that decouples consolidation from online inference. On LONGMEMEVAL and LoCoMo, using GPT and Qwen backbones, **LightMem** consistently surpasses strong baselines, improving QA accuracy by up to 7.7% / 29.3%, reducing total token usage by up to $38\times$ / $20.9\times$ and API calls by up to $30\times$ / $55.5\times$, while purely online test-time costs are even lower, achieving up to $106\times$ / $117\times$ token reduction and $159\times$ / $310\times$ fewer API calls. The code is available at <https://github.com/zjunlp/LightMem>.

1 INTRODUCTION

Memory is fundamental to intelligent agent, enabling the assimilation of prior experiences, contextual cues, and task-specific knowledge that underpin robust reasoning and decision-making (Wang et al., 2024; Behrouz et al., 2024; Du et al., 2025; Zhang et al., 2024). While Large Language Models (LLMs) (DeepSeek-AI et al., 2025; Achiam et al., 2023) demonstrate remarkable capabilities across a wide range of tasks, they exhibit significant limitations when engaged in long-context or multi-turn interaction scenarios due to fixed context windows and the “lost in the middle” problem (Liu et al., 2024). Memory systems are pivotal for overcoming these limitations, as they allow LLMs to maintain a persistent state across extended interactions. Recent works (Li et al., 2025b; Yang et al., 2024; Chhikara et al., 2025; Kang et al., 2025) address this challenge by building explicit external memory through sequential summarization and long term storage, enabling models to retain and retrieve relevant information over long horizons.

Note that a typical LLM memory system processes raw interaction data into manageable chunks, such as turn- or session-level in dialogue scenarios (Xu et al., 2025; Li et al., 2025a), organizes them into long-term memory (e.g., databases or knowledge graphs) by indexing them into memory units, and continuously updates by adding new information and discarding outdated or conflicting content (Zhong et al., 2024). This enables retrieval of relevant memories, improving coherence, and personalization in long-context, multi-turn scenarios.

*Corresponding Author.

LIGHTMEM: 轻量级高效内存增强生成

方继瞻[♣], 邓新乐[♣], 许浩明[♣], 蒋子岩[♣], 唐宇琪[♣], 许子文[♣], 邓淑敏[◇], 姚云芝[♣], 王梦如[♣], 乔舒菲[♣], 陈华俊[♣], 张
宁宇^{♣♣*}
[♣]浙江大学 [◇]新加坡国立大学 [♣]南京大学新型软件技术国家重点实验室, 中
国南京{fangjizhan, zhangningyu}@zju.edu.cn

摘要

尽管大语言模型（LLMs）能力出众，但在动态复杂环境中难以有效利用历史交互信息。内存系统能够通过引入持久信息存储、检索和利用机制，使LLMs超越无状态交互。然而，现有内存系统往往带来显著的时间和计算开销。为此，我们提出了一种新的内存系统，称为**LightMem**，在内存系统的性能和效率之间取得平衡。受人类记忆的阿特金森-希夫林模型启发，**LightMem** 将内存组织为三个互补的阶段。首先，受认知启发的感官记忆通过轻量级压缩快速过滤无关信息，并根据主题对信息进行分组。接下来，主题感知的短期记忆整合这些基于主题的分组，对内容进行组织和总结，以实现更结构化的访问。最后，带睡眠时间更新的长期记忆采用离线流程，将整合与在线推理解耦。在LONGMEMEVAL 和LoCoMoO，使用GPT和Qwen主干网络，LightMem持续超越强基线，将问答准确率提高至7.7%/29.3%，减少总token使用量至 $38\times$ / $20.9\times$ ，并减少API调用次数至 $30\times$ / $55.5\times$ ，而纯在线测试时成本更低，实现高达 $106\times$ / $117\times$ token 减少和 $159\times$ / $310\times$ 更少的API调用。代码可在 <https://github.com/zjunlp/LightMem> 获取。

1 引言

内存是智能体（intelligent agent）的基础，使其能够同化先前的经验、上下文线索和特定任务的知识，从而支撑强大的推理和决策能力（Wang 等人，2024；Behrouz 等人，2024；Duet 等人，2025；Zhang 等人，2024）。尽管大语言模型（LLMs）（DeepSeek-AI 等人，2025；Achiam 等人，2023）在广泛任务中展现出卓越的能力，但由于固定的上下文窗口和“中间丢失”问题（Liu 等人，2024），它们在长上下文或多轮交互场景中表现出显著的局限性。内存系统能够克服这些局限性，因为它们允许大语言模型在扩展交互中保持持续状态。近期研究（Li 等人，2025b；Yang 等人，2024；Chhikara 等人，2025；Kang 等人，2025）通过顺序摘要和长期存储构建显式的外部内存来应对这一挑战，使模型能够在长时程内保留和检索相关信息。

请注意，典型的大语言模型内存系统会将原始交互数据处理成可管理的数据块，例如在对话场景中按回合或会话级别（Xu 等人,2025; Li 等人, 2025a），通过将其索引到内存单元中组织成长期记忆（例如数据库或知识图谱），并通过添加新信息以及丢弃过时或冲突的内容进行持续更新（Zhong 等人, 2024）。这使得能够检索相关记忆，提高长上下文、多回合场景中的连贯性和个性化。

*通讯作者。

Challenges. Despite these advances, as shown in Figure 1, contemporary memory systems still suffer from significant inefficiencies and consistency issues. First, in long interactions (e.g., dialogue scenarios), both user inputs and model responses often contain substantial redundant information (Maharana et al., 2024; Wu et al., 2025). Such information is typically irrelevant to downstream tasks or subsequent memory construction, and in some cases, may even negatively affect the model’s in-context learning capability (Liu et al., 2023; Pan et al., 2025). However, current mainstream memory-related studies generally process the raw information directly without any filtering or refinement, leading to high overhead from noisy or irrelevant data. This inflates token consumption without proportional gains in reasoning quality or coherence. Second, memory construction typically **treats each turn in isolation or relies on rigid context-window boundaries**, failing to model semantic connections across different turns (Tan et al., 2025). As a result, during subsequent memory item construction, the backbone LLM may generate inaccurate or incomplete item representations due to overly entangled topics or semantics, leading to the loss of crucial contextual details. Third, memory updates and forgetting are usually performed directly **during inference and task execution**. This tight coupling introduces long test-time latency in long-horizon tasks and prevents deeper, reflective processing of past experiences.

Building Lightweight Memory. Inspired by the efficiency and structure of human memory, we introduce **LightMem**. In particular, LightMem emulates human memory through three key components: (1) A *pre-compression sensory memory module* that filters redundant or low-value tokens from raw input and buffers the distilled content. (2) A *topic-aware short-term memory* that leverages semantic and topical similarity to dynamically group related utterances into coherent segments. By adaptively determining segment boundaries based on content instead of fixed window sizes, this module produces more concentrated and meaningful memory units. This not only reduces the frequency of memory construction but also enables more precise and efficient retrieval during inference. (3) A *sleep-time update* mechanism for long-term memory maintenance. New memory entries are initially stored with timestamps to support immediate (“soft”) updates for real-time responsiveness. Later, during designated offline periods (i.e., “sleep”), the system reorganizes, de-duplicates, and abstracts these entries, resolving inconsistencies and strengthening cross-knowledge connections. Crucially, this decouples expensive memory maintenance from real-time inference, enabling reflective, high-fidelity updates without introducing latency. By systematically filtering, organizing, and consolidating relevant information, LightMem substantially reduces computational overhead and API costs while sustaining accurate, coherent reasoning over extended interactions. We detail each component in §3.

Results and Evaluation. On LongMemEval (Wu et al., 2025), LightMem consistently outperforms the strongest baseline, improving accuracy by 2.09%–6.40% with GPT and up to 7.67% with Qwen. In terms of overall efficiency (online + offline), LightMem reduces total token usage by up to $38\times$ for GPT and $21.8\times$ for Qwen, lowers API calls by up to $30\times$ and $17.1\times$, and accelerates runtime by up to $12.4\times$ and $6.3\times$, respectively. If considering only online test-time costs, the gains become even larger: LightMem cuts token usage by up to $105.9\times$ (GPT) and $117.1\times$ (Qwen), and reduces API calls by up to $159.4\times$ and $309.9\times$. On the LoCoMo benchmark (Maharana et al., 2024), LightMem maintains strong advantages, achieving 6.10%–29.29% higher accuracy and substantial efficiency improvements—boosting token efficiency by up to $20.92\times$, reducing API calls by up to $55.48\times$, and speeding up runtime by up to $8.21\times$ across GPT and Qwen backbones. Furthermore, case studies in §5.6 show that the offline “sleep-time” consolidation enhances long-term memory reliability, mitigating information loss.

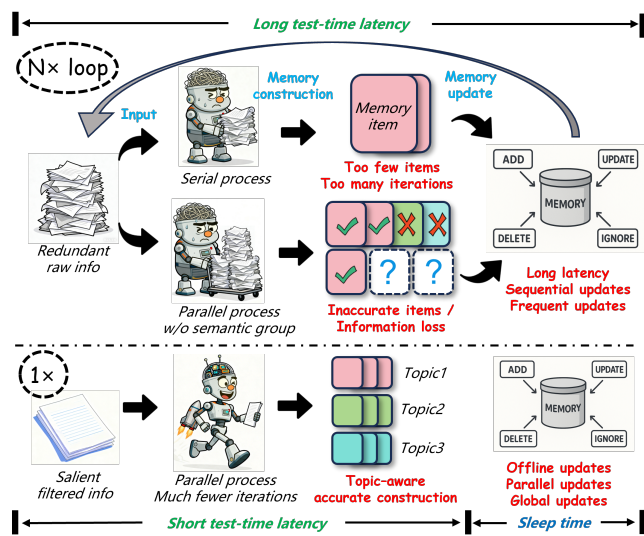


Figure 1: Comparison of previous works and **LightMem**.

挑战。 尽管取得了这些进展，如图 1 所示，当代内存系统仍然存在显著的低效性和一致性问题。首先，在长交互（例如对话场景）中，用户输入和模型响应通常包含大量冗余信息（Maharana 等人, 2024; Wu 等人, 2025）。此类信息通常与下游任务或后续记忆构建无关，在某些情况下甚至可能对模型的情境学习能力产生负面影响（Liu 等人, 2023; Pan 等人, 2025）。然而，当前主流的内存相关研究通常直接处理原始信息，没有任何过滤或精炼，导致从噪声或无关数据中产生高开销。这导致 token 消耗增加，但推理质量或连贯性的提升不成比例。其次，记忆构建通常 **将每个回合孤立处理或依赖僵化的上下文窗口边界**，未能建模不同回合之间的语义连接（Tan 等人, 2025）。因此，在后续记忆项构建过程中，由于主题或语义过度纠缠，骨干大语言模型可能生成不准确或不完整的项表示，导致关键上下文细节的丢失。第三，记忆更新和遗忘通常在 **推理和任务执行过程中直接进行**。这种紧密耦合导致长时程任务中的测试时延迟，并阻碍对过去经验的更深入、更具反思性的处理。

构建轻量级内存。 受人类记忆的效率 and 结构启发，我们引入 **LightMem**。具体而言，LightMem 通过三个关键组件模拟人类记忆：(1) 一个预压缩感觉记忆模块，用于从原始输入中过滤冗余或低价值 token，并缓冲提炼后的内容。(2) 一个主题感知短期记忆，利用语义和主题相似性动态地将相关话语分组为连贯的片段。通过基于内容而非固定窗口大小自适应地确定片段边界，该模块生成更集中、更有意义的内存单元。这不仅减少了内存构建的频率，还实现了更

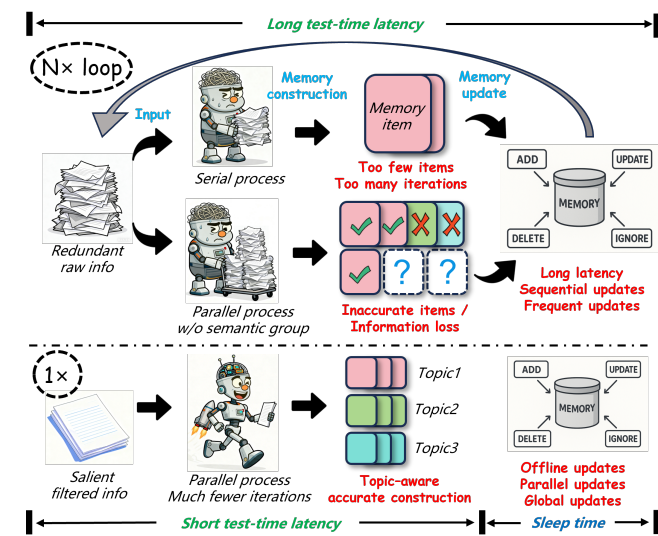


图1：与以往工作及**LightMem**的比较。

精确高效的推理检索。(3) 一个睡眠时更新机制用于长期记忆维护。新内存条目最初存储带时间戳，以支持即时（“软”）更新以实现实时响应。之后，在指定的离线时段（即“睡眠”期间），系统重新组织、去重和抽象这些条目，解决不一致性并加强跨知识连接。关键在于，这种解耦了昂贵的内存维护与实时推理，使系统能够在不引入延迟的情况下进行反思性、高保真度的更新。通过系统性地过滤、组织和整合相关信息，LightMem 大幅降低了计算开销和 API 成本，同时保持了跨长时间交互的准确、连贯推理。我们在 §3 中详细说明每个组件。

结果与评估。 在 LongMemEval (Wu 等人, 2025) 上，LightMem 始终优于最强的基线，在使用 GPT 时准确率提升了 2.09%–6.40%，在使用 Qwen 时最高提升了 7.67%。在整体效率（在线/离线）方面，LightMem 将 GPT 的总 token 使用量减少高达 $38\times$ ，Qwen 的总 token 使用量减少 $21.8\times$ ，API 调用次数减少高达 $30\times$ 和 $17.1\times$ ，运行时间分别加速高达 $12.4\times$ 和 $6.3\times$ 。如果仅考虑在线测试时间成本，收益会更大：LightMem 将 GPT 的 token 使用量减少高达 $105.9\times$ (GPT) 和 $117.1\times$ (Qwen)，API 调用次数减少高达 $159.4\times$ 和 $309.9\times$ 。在 LoCoMo 基准 (Maharana 等人, 2024) 上，LightMem 保持了显著优势，准确率提升了 6.10%–29.29%，并实现了显著的效率改进——token 效率提升高达 $20.92\times$ ，API 调用次数减少高达 $55.48\times$ ，运行时间加速高达 $8.21\times$ ，涵盖 GPT 和 Qwen 主干。此外，§5.6 中的案例研究表明，离线“睡眠时间”整合增强了长期记忆的可靠性，减轻了信息丢失。

2 PRELIMINARY

2.1 CONVENTIONAL MEMORY SYSTEMS FOR LLMs

We describe mainstream memory architectures pipeline in terms of two major stages. **(I) Memory Bank Construction.** This stage can be further decomposed into three sub-stages: (a) Raw data D are first processed at a chosen level of granularity, $D^{(g)} = f_{\text{seg}}(D; g), g \in \{\text{turn, session, topic}\}$ in dialog scenario; (b) The segmented data $D^{(g)}$ are then summarized or extracted to generate memory entries, $E = f_{\text{sum}}(D^{(g)})$, which are stored and organized within structural backends such as vector databases or knowledge graphs to enable long-term retention; (c) Many systems incorporate an updating mechanism to mitigate issues such as context conflicts or outdated information, $M' = f_{\text{update}}(M, R; U)$, where M denotes the existing memory bank, R represents newly generated memory entries, and U specifies the update or forgetting policy. **(II) Retrieval and Usage.** When a new user query arrives, the system retrieves relevant entries from the memory bank, integrates them with the query to construct the final prompt, and then invokes the model to produce a response.

2.2 LIMITATIONS OF EXISTING LLM MEMORY SYSTEMS

Compared to human memory, current LLM memory systems are burdened by high maintenance costs, mainly due to three limitations: **1) Redundant Sensory Memory.** In current systems, $f_{\text{sum}}()$ and $f_{\text{gran}}(; g = \text{topic})$ are typically executed by calling stronger LLMs. Feeding raw data D directly wastes resources and even weakens in-context learning due to redundancy. A key challenge is to design lightweight mechanisms that pre-compress inputs and apply pre-attention strategies to capture semantic units at different granularities efficiently. **2) Balancing Effectiveness and Efficiency in STM.** As shown in Figure 1, when input granularity is fixed, $D^{(g)}$ must pass through the entire pipeline. Excessively fine granularity increases latency and underutilizes STM capacity, whereas overly coarse granularity without semantic constraints or grouping may cause mixed or entangled semantics and topics, leading to inaccurate memory construction and loss of fine-grained details in subsequent processes. This calls for strategies that better balance effectiveness and efficiency in STM. **3) Inefficient LTM Updating.** Current $f_{\text{update}}()$ mechanisms face two main issues: (i) enforcing strict real-time updates at test time incurs significant latency, whereas STM can provide short-term context without immediate LTM updates; (ii) memory banks are updated sequentially due to ordering constraints (read-after-write/write-after-read), rather than being triggered dynamically. These limitations raise a research question: *Can we design LLM memory that is both efficient and lightweight, inspired by human memory mechanisms?*

3 LIGHTMEM ARCHITECTURE

Analogous to the human memory, we design LightMem as shown in Figure 2, which consists of three light modules: *Light1* implements an efficient *Sensory Memory Module* that selectively preserves salient information from raw input (§3.1), *Light2* realizes a topic-aware *STM Module* for transient information processing (§3.2), and *Light3* provides an *LTM module* designed to minimize test time update latency (§3.3) with a sleep time update mechanism. The overall pipeline framework of **LightMem**, its specific models, and comparisons with other memory frameworks are presented in Appendix A.1. The complexity analysis for LightMem’s efficiency gains is in Section 4.

3.1 LIGHT1: COGNITIVE-INSPIRED SENSORY MEMORY

In long horizon interaction scenarios, such as user–assistant dialogues, a large portion of the information is redundant. Therefore, we design a *Pre-Compressing Submodule* to eliminate redundant tokens, followed by the *Topic Segmentation Submodule* that forms semantic topic-based segments for following faster and more accurate memory construction.

Pre-Compressing Submodule. This module leverages a compression model θ to eliminate redundant tokens, tailored for compatibility with the downstream memory construction phase:

$$\hat{\mathbf{x}} = \{x_i \in \mathbf{x} \mid P(\text{retain } x_i \mid \mathbf{x}; \theta) > \tau\}, \tau = \text{Percentile}(\{x_j\}, r),$$

Following Xia et al. (2025), we use LLMingua-2 (Pan et al., 2024b) as our compression model θ . Let $\mathbf{x}$ be the raw input tokens, θ the model, and r the compression ratio. The threshold τ is set to

2 预备知识

2.1 大语言模型的传统内存系统

我们按两个主要阶段描述主流记忆架构的流程。 **(I) 内存库构建。** 此阶段可进一步分解为三个子阶段：(a) 原始数据 D 首先在选定的粒度级别进行处理， $D^{(g)} = f_{\text{seg}}(D; g), g \in \{\text{转}, \text{会话}, \text{主题}\}$ 在对话场景中；(b) 分段数据 $D^{(g)}$ 然后被总结或提取以生成内存条目， $E = f_{\text{sum}}(D^{(g)})$ ， 这些条目存储并组织在向量数据库或知识图谱等结构化后端中， 以实现长期保留；(c) 许多系统包含更新机制以缓解上下文冲突或过时信息等问题, $M' = f_{\text{update}}(M, R; U)$ ， 其中 M 表示现有内存库， R 代表新生成的内存条目， U 指定更新或遗忘策略。 **(II) 检索和使用。** 当收到新的用户查询时， 系统从内存库中检索相关条目， 将它们与查询整合以构建最终提示， 然后调用模型生成响应。

2.2 现有LLM内存系统的局限性

与人类记忆相比， 当前的LLM内存系统由于三个局限性而负担着高昂的维护成本： **1) 冗余的感官记忆。** ()和 $f_{\text{gran}}(; g = \text{主题})$ 通常由更强的LLM执行。 直接输入原始数据浪费资源， 甚至由于冗余而削弱情境学习。 一个关键挑战是设计轻量级机制， 预先压缩输入并应用预注意力策略， 以高效地捕获不同粒度的语义单元。 **2) 在STM中平衡有效性和效率。** 如图1所示， 当输入粒度固定时， $D^{(g)}$ 必须通过整个管道。 过细的粒度会增加延迟并浪费STM容量， 而缺乏语义约束或分组而过粗的粒度可能导致混合或纠缠的语义和主题， 从而导致后续过程中记忆构建不准确和细粒度细节的丢失。 这需要更好的平衡STM中的有效性和效率的策略。 **3) LTM更新效率低下。** 当前的 $f_{\text{update}}()$ 机制面临两个主要问题： (i) 在测试时强制实时更新会产生显著延迟， 而STM可以在没有立即LTM更新的情况下提供短期上下文； (ii) 由于顺序约束（读后写/写后读）， 内存库按顺序更新， 而不是动态触发。 这些局限性提出了一个研究问题： 我们能设计出既高效又轻量级的LLM内存， 借鉴人类记忆机制吗？

3 LightMem 架构

类似于人类记忆， 我们设计了如图2所示的LightMem， 它由三个轻量级模块组成： *Light1* 实现了一个高效的感官记忆模块， 用于从原始输入中选择性保留显著信息 (§ 3.1)， *Light2* 实现了一个主题感知的短期记忆模块， 用于处理短暂信息 (§ 3.2)， 而*Light3* 提供了一个长期记忆模块， 该模块设计用于最小化测试时间更新延迟 (§ 3.3)， 并采用睡眠时间更新机制。 LightMem的整体管道框架、 其特定模型以及与其他记忆框架的比较将在附录A.1中介绍。 LightMem效率提升的复杂度分析在4节。

3.1 LIGHT1: 认知启发式感官记忆

在长时程交互场景中， 例如用户-助手对话， 大量信息是冗余的。 因此， 我们设计了一个预压缩子模块来消除冗余标记， 然后是主题分割子模块， 它形成基于语义的主题片段， 以便后续更快、 更准确地构建记忆。

预压缩子模块。 该模块利用压缩模型 θ 消除冗余标记， 专为与下游记忆构建阶段兼容而设计：

$$\hat{\mathbf{x}} = \{x_i \in \mathbf{x} \mid P(\text{retain } x_i \mid \mathbf{x}; \theta) > \tau\}, \tau = \text{Percentile}(\{x_j\}, r),$$

随后 夏等人 (2025), 我们使用 LLMingua-2 (潘等人。 , 2024b) 作为我们的压缩模型 θ 。 令 $\mathbf{x}$ 为原始输入标记， θ 模型， 和 r 压缩率。 阈值 τ 设置为

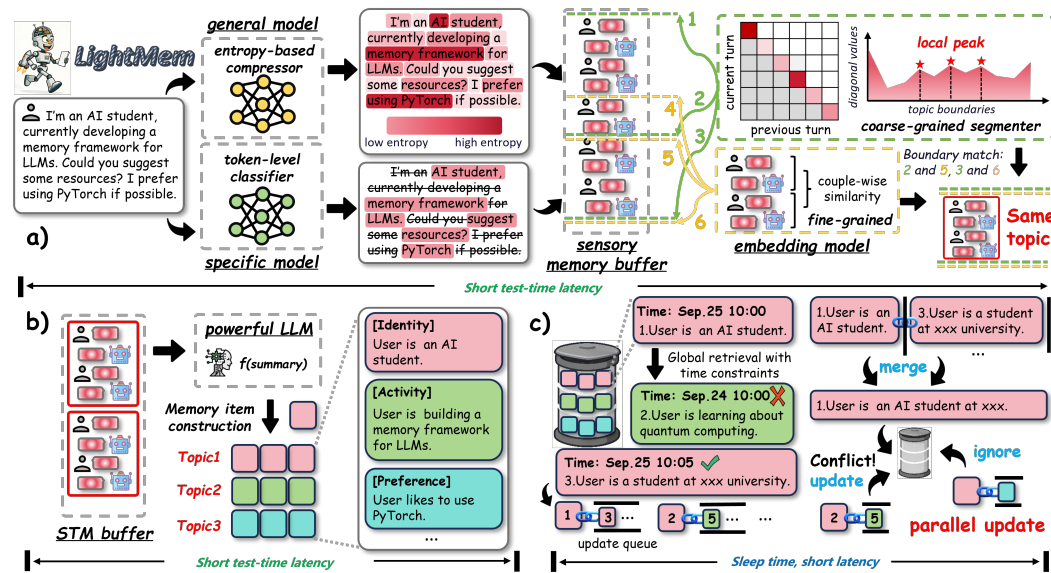


Figure 2: The **LightMem** architecture. **LightMem** consists of three modules: a) An efficient *Sensory Memory Module*, b) a topic aware *STM Module*, and c) an *LTM module* updated in sleep time.

the r -th percentile of retention scores, keeping only tokens above τ . For $P(\text{retain } x_i | \mathbf{x})$, we treat the compression process as a binary token classification task (“retain” or “discard”). For each token x_i in a sequence $\mathbf{x}$, the model θ outputs a logit vector ℓ_i , and the retention probability is given by:

$$P(\text{retain } x_i | \mathbf{x}; \theta) = \text{softmax}(\ell_i)_1,$$

where the subscript 1 denotes the “retain” class. Tokens with probabilities above a dynamic threshold are included in the compressed sequence. In addition, **LightMem** can also employ more general generative LLM as the pre-compression model. We further implement a token filtering mechanism based on the cross-entropy between the model’s predicted distribution and the true token labels:

$$P(\text{retain } x_i | \mathbf{x}; \theta) = - \sum_{x_i \in \mathcal{V}} q(x_i) \log P(x_i | \mathbf{x}; \theta)$$

where $q(x_i)$ denotes the true token label distribution. Tokens with higher conditional entropy under a given context are more uncertain and less predictable, indicating greater informational uniqueness and a more critical role in semantic expression, such distinctive tokens are essential for subsequent memory construction and are therefore retained.

Topic Segmentation Submodule. Existing works indicate that topic-granular input facilitates improved performance in memory systems (Pan et al., 2025; Tan et al., 2025). As shown in Figure 2, **LightMem** maintains a sensory memory buffer to temporarily store information after pre-compression. When the accumulated information reaches the buffer’s maximum capacity, a hybrid topic segmentation operation based on attention and similarity is triggered. We use the compression model θ and an embedding model to compute attention matrices and semantic similarities, respectively. We define the final segmentation boundaries as the intersection of attention-based boundaries $\mathcal{B}_1$ and similarity-based boundaries $\mathcal{B}_2$:

$$\mathcal{B}_1 = \{k | M_{k,k-1} > M_{k-1,k-2}, M_{k,k-1} > M_{k+1,k}, 1 < k < n\},$$

$$\mathcal{B}_2 = \{k | \text{sim}(s_{k-1}, s_k) < \tau, 1 \leq k < n\}, \quad \mathcal{B} = \mathcal{B}_1 \cap \mathcal{B}_2.$$

Specifically, dialogue scenarios possess natural semantic units, namely the conversational turn. We construct a turn-level attention matrix $M \in \mathbb{R}^{n \times n}$. $\mathcal{B}_1$ are identified as local maxima in the sequence $\{M_{k,k-1}\}$, i.e., the sub-diagonal elements of M corresponding to attention between consecutive sentences. The detailed process of $\mathcal{B}_1$ and illustrative cases are provided in Appendix C.1. To mitigate

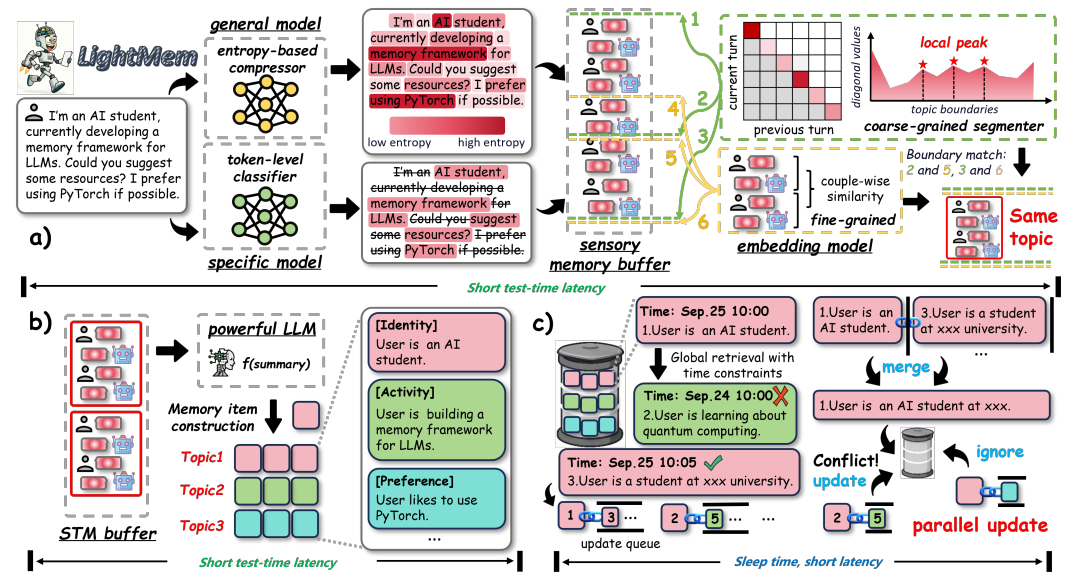


Figure 2: The **LightMem** architecture. **LightMem** consists of three modules: a) An efficient *Sensory Memory Module*, b) a topic aware *STM Module*, and c) an *LTM module* updated in sleep time.

the r -th percentile of retention scores, keeping only tokens above τ . For $P(\text{retain } x_i | \mathbf{x})$, we treat the compression process as a binary token classification task (“retain” or “discard”). For each token x_i in a sequence $\mathbf{x}$, the model θ outputs a logit vector ℓ_i , and the retention probability is given by:

$$P(\text{retain } x_i | \mathbf{x}; \theta) = \text{softmax}(\ell_i)_1,$$

where the subscript 1 denotes the “retain” class. Tokens with probabilities above a dynamic threshold are included in the compressed sequence. In addition, **LightMem** can also employ more general generative LLM as the pre-compression model. We further implement a token filtering mechanism based on the cross-entropy between the model’s predicted distribution and the true token labels:

$$P(\text{retain } x_i | \mathbf{x}; \theta) = - \sum_{x_i \in \mathcal{V}} q(x_i) \log P(x_i | \mathbf{x}; \theta)$$

where $q(x_i)$ denotes the true token label distribution. Tokens with higher conditional entropy under a given context are more uncertain and less predictable, indicating greater informational uniqueness and a more critical role in semantic expression, such distinctive tokens are essential for subsequent memory construction and are therefore retained.

Topic Segmentation Submodule. Existing works indicate that topic-granular input facilitates improved performance in memory systems (Pan et al., 2025; Tan et al., 2025). As shown in Figure 2, **LightMem** maintains a sensory memory buffer to temporarily store information after pre-compression. When the accumulated information reaches the buffer’s maximum capacity, a hybrid topic segmentation operation based on attention and similarity is triggered. We use the compression model θ and an embedding model to compute attention matrices and semantic similarities, respectively. We define the final segmentation boundaries as the intersection of attention-based boundaries $\mathcal{B}_1$ and similarity-based boundaries $\mathcal{B}_2$:

$$\mathcal{B}_1 = \{k | M_{k,k-1} > M_{k-1,k-2}, M_{k,k-1} > M_{k+1,k}, 1 < k < n\},$$

$$\mathcal{B}_2 = \{k | \text{sim}(s_{k-1}, s_k) < \tau, 1 \leq k < n\}, \quad \mathcal{B} = \mathcal{B}_1 \cap \mathcal{B}_2.$$

Specifically, dialogue scenarios possess natural semantic units, namely the conversational turn. We construct a turn-level attention matrix $M \in \mathbb{R}^{n \times n}$. $\mathcal{B}_1$ are identified as local maxima in the sequence $\{M_{k,k-1}\}$, i.e., the sub-diagonal elements of M corresponding to attention between consecutive sentences. The detailed process of $\mathcal{B}_1$ and illustrative cases are provided in Appendix C.1. To mitigate

attention sinks and dilution in attention-based methods, we compute semantic similarity between adjacent turns near each candidate boundary in $\mathcal{B}_1$. Boundaries with similarity below threshold τ form set $\mathcal{B}_2$, which helps determine the final topic boundaries $\mathcal{B}$.

3.2 LIGHT2: TOPIC-AWARE SHORT-TERM MEMORY

After obtaining individual topic segments, forming an index structure of {topic, message turns}, where message turns = $\{user_i, model_i\}$. These are first placed into the STM buffer. When the token count in the buffer reaches a preset threshold, we invoke LLM f_{sum} to generate concise summaries of every structure. The final index structure stored in LTM is {topic, $\{sum_i, user_i, model_i\}$ }.

$$\text{sum}_i = f_{\text{sum}}(S_i), \quad S_i \subseteq \{user_i, model_i\}, \quad S_i \neq \emptyset,$$

$$\text{Entry}_i = \{\text{topic}, \mathbf{e}_i := \text{embedding}(\text{sum}_i), user_i, model_i\},$$

where Entry_i denotes the memory entry to be stored in LTM. Compared with inputting at the granularity of a single turn or session, directly feeding multiple sessions can reduce subsequent API calls but often introduces inaccurate memory entries due to excessive topic mixing, leading to performance degradation. In contrast, topic-constrained input granularity minimizes API calls to the greatest extent while preserving summarization accuracy and maintaining stable system performance.

3.3 LIGHT3: LONG-TERM MEMORY WITH SLEEP-TIME UPDATE

Soft Updating at Test Time. At test time, when memory entries arrive, LightMem directly inserts them into LTM with soft updates, thereby decoupling the update process from online inference. Due to real-time updates being converted to direct insertions, interaction latency is significantly reduced. After all entries are inserted or when an update trigger arrives, we compute an update queue for every entry in LTM.

$$\mathcal{Q}(e_i) = \text{Top}_k \left\{ (e_j, \text{sim}(v_i, v_j)) \mid t_j \geq t_i, j \neq i \right\}_{:n},$$

where e_i denotes the i -th memory entry with embedding v_i and timestamp t_i , $\text{sim}(\cdot, \cdot)$ is the similarity function, and $\text{Top}_k\{\cdot\}_{:n}$ indicates selecting the top- k most similar candidates, with the update queue $\mathcal{Q}(e_i)$ length fixed at n . Consistent with existing work, we select the top- k existing memory entries with the highest semantic similarity as potential update sources. On this basis, we further impose the constraint that only entries with later timestamps are allowed to update earlier ones ($t_j \geq t_i$), which is consistent with realistic temporal dynamics. Here, $\mathcal{Q}(e_i)$ denotes the queue of other entries that may update e_i . Since this process involves only similarity retrieval, it is fast and lightweight, and can be executed offline in parallel with online inference.

Offline Parallel Update. LightMem does not simply transfer online update latency to offline phases, it substantially reduces the overall update latency. The online update mechanism in existing memory frameworks enforces sequential updates, leading to a total latency that accumulates with each update. As shown in Figure 2, in LightMem, each memory entry maintains a global update queue, with each queue corresponding to a distinct f_{update} operation. Since the update targets are independent across queues, updates can be executed in parallel, thereby greatly reducing the total latency.

4 COMPLEXITY ANALYSIS ABOUT LIGHTMEM

Method	Summary Tokens	Update Tokens	API Calls	Runtime
Baselines	$N(L_{\text{sum-in}} + T + L_{\text{sum-out}})$	$NM_1R_1(L_{\text{up-in}} + L_{\text{up-out}})$	N	$O(N)$
LightMem	$\frac{Nr^xT}{th}(L_{\text{sum-in}} + th + L_{\text{sum-out}})$	$\frac{Nr^xT}{th}M_2R_2(L_{\text{up-in}} + L_{\text{up-out}})$	$\frac{Nr^xT}{th}$	$O\left(\frac{Nr^xT}{th}\right)$

Table 1: Complexity comparison between LightMem and other memory systems. The specific definitions of each symbol are provided in the Appendix A.2.

As shown in Table 4, we consider a dialogue with N turns, each containing on average T tokens. In conventional memory systems, each turn triggers a summarization call, consuming $L_{\text{sum-in}} + T + L_{\text{sum-out}}$ tokens and totaling $N(L_{\text{sum-in}} + T + L_{\text{sum-out}})$ tokens with N API calls. Each summarization

基于注意力方法中的注意力沉没和注意力稀释问题，我们在 $\mathcal{B}_1$ 中计算相邻回合在候选边界附近的语义相似度。相似度低于阈值 τ 的边界形成集合 $\mathcal{B}_2$ ，这有助于确定最终的主题边界 $\mathcal{B}$ 。

3.2 LIGHT2: 主题感知短期记忆

在获取单个主题片段后，构建一个索引结构，即 {主题，消息转换}，其中消息转换 = $\{user_i, model_i\}$ 。这些首先被放入STM缓冲区。当缓冲区中的令牌数量达到预设阈值时，我们调用LLM f_{sum} 来为每个结构生成简洁的摘要。最终存储在LTM中的索引结构是 {主题， $\{sum_i, user_i, model_i\}$ }。

$$\text{sum}_i = f_{\text{sum}}(S_i), \quad S_i \subseteq \{user_i, model_i\}, \quad S_i \neq \emptyset,$$

$$\text{Entry}_i = \{\text{topic}, \mathbf{e}_i := \text{embedding}(\text{sum}_i), user_i, model_i\},$$

其中 Entry_i 表示要存储在LTM中的内存条目。与在单个回合或会话的粒度下输入相比，直接输入多个会话可以减少后续API调用，但由于过多的主题混合，经常引入不准确的内存条目，导致性能下降。相比之下，主题约束的输入粒度在最大限度地减少API调用的同时，保留了摘要的准确性，并保持了系统性能的稳定。

3.3 LIGHT3: 带睡眠时间更新的长期记忆

测试时软更新。在测试时，当内存条目到达时，LightMem 直接使用软更新将它们插入LTM，从而将更新过程与在线推理解耦。由于实时更新被转换为直接插入，交互延迟显著降低。在所有条目插入完毕或当更新触发器到达时，我们为LTM中的每个条目计算一个更新队列。

$$\mathcal{Q}(e_i) = \text{Top}_k \left\{ (e_j, \text{sim}(v_i, v_j)) \mid t_j \geq t_i, j \neq i \right\}_{:n},$$

其中 e_i 表示第 i 个内存条目，具有嵌入 v_i 和时间戳 t_i ， $\text{sim}(\cdot, \cdot)$ 是相似度函数，而 $\text{Top}_k\{\cdot\}_{:n}$ 表示选择最相似的 k 个候选，更新队列 $\mathcal{Q}(e_i)$ 的长度固定为 n 。与现有工作一致，我们选择语义相似度最高的前 k 个现有内存条目作为潜在的更新来源。在此基础上，我们进一步施加约束，即只有时间戳较晚的条目才允许更新时间戳较早的条目 ($t_j \geq t_i$)，这符合现实的时间动态。这里， $\mathcal{Q}(e_i)$ 表示可能更新 e_i 的其他条目的队列。由于该过程仅涉及相似度检索，因此它快速且轻量，可以离线并行执行，与在线推理同步。

离线并行更新。LightMem不会将在线更新延迟简单地转移到离线阶段，而是显著降低了整体更新延迟。现有内存框架中的在线更新机制强制顺序更新，导致总延迟随着每次更新而累积。如图2所示，在LightMem中，每个内存条目维护一个全局更新队列，每个队列对应一个不同的 f_{update} 操作。由于更新目标在队列之间是独立的，因此可以并行执行更新，从而大大降低了总延迟。

关于LightMem的复杂度分析

方法	摘要条目	更新条目	API调用运行时	
基线	$N(L_{\text{sum-in}} + T + L_{\text{sum-out}})$	$NM_1R_1(L_{\text{up-in}} + L_{\text{up-out}})$	N	$O(N)$
LightMem	$\frac{Nr^xT}{th}(L_{\text{sum-in}} + th + L_{\text{sum-out}})$	$\frac{Nr^xT}{th}M_2R_2(L_{\text{up-in}} + L_{\text{up-out}})$	$\frac{Nr^xT}{th}$	$O\left(\frac{Nr^xT}{th}\right)$

表1: LightMem与其他内存系统的复杂度比较。每个符号的具体定义见附录A.2。

如表4所示，我们考虑一个包含 N 轮对话，每轮平均包含 T 个token。在传统内存系统中，每轮对话会触发一次摘要调用，消耗 $L_{\text{sum-in}} + T + L_{\text{sum-out}}$ 个token，总共需要 $N(L_{\text{sum-in}} + T + L_{\text{sum-out}})$ 个token，并产生 N 次API调用。每次摘要

produces M_1 memory entries, a fraction R_1 of which retrieve at least one relevant neighbor and trigger an update, resulting in an update-token cost of $N M_1 R_1 (L_{\text{up-in}} + L_{\text{up-out}})$.

In **LightMem**, each turn is first passed through iterative pre-compression submodule, retaining only $r^x T$ tokens after x iterations, and appended to a short-term memory (STM) buffer of capacity th . Summarization is triggered only when the buffer reaches capacity, yielding $\frac{N r^x T}{th}$ summarization calls, each consuming $L_{\text{sum-in}} + th + L_{\text{sum-out}}$ tokens. Each summarization produces M_2 memory entries, but stricter retrieval constraints, including semantic similarity and timestamp filtering, reduce the fraction R_2 that trigger updates. Hence, the update phase involves $\frac{N r^x T}{th} M_2 R_2$ calls, with a total token cost of $\frac{N r^x T}{th} M_2 R_2 (L_{\text{up-in}} + L_{\text{up-out}})$.

Overall, **LightMem** requires only $\frac{N r^x T}{th}$ API calls for both summarization operations, substantially reducing token usage and call frequency compared to other systems. Correspondingly, the runtime complexity of other memory systems is $O(N)$, while LightMem achieves a reduced runtime of $O\left(\frac{N r^x T}{th}\right)$, reflecting the efficiency gain from compressed summarization and selective updates.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Experimental Details. (1) Our experiments adopt a realistic *Incremental Dialogue Turn Feeding* setting, where the entire dialogue history is fed and processed **at the turn level, one turn at a time**. This reflects practical scenarios where interactions between user and model is incrementally formed turn by turn. (Hu et al., 2025). (2) For considerations of both efficiency and effectiveness, we employ LLMingua-2 as our pre-compressor throughout all subsequent experiments. (3) The attention scores for topic segmentation are also obtained using LLMingua-2, the size of the sensory memory buffer is 512 tokens. All specific models used in this paper, can be found in Table 5.

Datasets & Baseline Methods. We use two well-known datasets, LONGMEMEVAL (Wu et al., 2025) (specifically the LongMemEval-S split) and Locomo (Maharana et al., 2024) to evaluate memory ability. We compare **LightMem** against several representative baselines of conversational memory modeling. ① *Full Text*, ② *Naive RAG*, ③ *LangMem* (LangChain, 2025), ④ *A-MEM* (Xu et al., 2025), ⑤ *MemoryOS* (Kang et al., 2025), ⑥ *Mem0* (Chhikara et al., 2025). In addition, all methods use GPT-4o-mini, Qwen3-30B-A3B-Instruct-2507 and GLM-4.6 as the LLM backbones. Details on dataset, baselines, and experimental settings are provided in the Appendix D.

Metrics. We evaluate these methods using both effectiveness and efficiency metrics. For effectiveness, we report **Accuracy (ACC)**, defined as the proportion of correctly answered questions. The evaluation is conducted with *GPT-4o-mini* as an LLM judge, guided by a detailed evaluation prompt (see Appendix E.1). For efficiency, we focus on tracking the computational costs of the LLM invocations in memory bank construction stage (see Section 2.1), all averaged across the entire dataset, as it is the one tied to the design and implementation differences of memory systems. The retrieval and usage stage is not our focus, because for fair comparison, The $f_{\text{retrieve}}()$, $f_{\text{chat}}()$ and number of retrieved entries are same among all methods. As a result, their costs exhibit only minor differences, and this stage is largely orthogonal to the design of memory systems, as shown in the table. Within the memory bank construction stage, only the two sub-processes **Summary** and **Update** involve the use of LLMs, $f_{\text{sum/extract}}()$ and $f_{\text{update}}()$. So for both processes, we report the token consumption from LLM calls, including input tokens, output tokens, and total token usage (in thousands). Additionally, we track **API Calls** counting the total number of LLM invocations, and **Runtime** recording the overall execution time for memory bank construction stage.

5.2 MAIN RESULTS

As shown in Table 2 and Table 3, **LightMem** demonstrates superior effectiveness and efficiency on both datasets across both GPT and Qwen backbones. For a fair comparison, all efficiency metrics for LightMem in the following analysis refer to the **combined online and offline** costs.

LongMemEval. On the LongMemEval benchmark, LightMem consistently outperforms the strongest baseline, A-Mem, in the ACC metric, improving accuracy by 2.09%–6.40% with GPT and

生成 M_1 个内存条目, 其中 R_1 的比例会检索至少一个相关邻居并触发更新, 导致更新token成本为 $N M_1 R_1 (L_{\text{up-in}} + L_{\text{up-out}})$ 。

在**LightMem**中, 每一轮首先通过迭代预压缩子模块进行处理, 经过 x 次迭代后仅保留 $r^x T$ 个 token, 并追加到容量为 th 的短期记忆 (STM) 缓冲区中。只有当缓冲区达到容量时才会触发摘要生成, 产生 $\frac{N r^x T}{th}$ 次摘要调用, 每次消耗 $L_{\text{sum-in}} + th + L_{\text{sum-out}}$ 个 token。每次摘要生成会产生 M_2 个内存条目, 但更严格的检索约束, 包括语义相似度和时间戳过滤, 会减少触发更新的 R_2 比例。因此, 更新阶段涉及 $\frac{N r^x T}{th} M_2 R_2$ 次调用, 总token成本为 $\frac{N r^x T}{th} M_2 R_2 (L_{\text{up-in}} + L_{\text{up-out}})$ 。

总体而言, **LightMem** 的摘要操作仅需 $\frac{N r^x T}{th}$ 次 API 调用, 与其他系统相比大幅减少了 token 使用和调用频率。相应地, 其他内存系统的运行时复杂度为 $O(N)$, 而 LightMem 实现了 $O\left(\frac{N r^x T}{th}\right)$ 的降低运行时, 这反映了压缩摘要和选择性更新带来的效率提升。

5 实验

5.1 实验设置

实验细节. (1) 我们的实验采用一个真实的增量对话轮次输入设置, 其中整个对话历史被输入和处理在**轮次级别, 一次一轮**。这反映了用户和模型之间交互在实际场景中逐轮增量形成的情况。(Hu 等人, 2025)。(2) 为了考虑效率和效果的平衡, 我们在所有后续实验中采用 LLMingua-2 作为预压缩器。(3) 主题分割的注意力分数也是使用 LLMingua-2 获得的, 感觉记忆缓冲区的大小为 512 个 token。本文中使用的所有具体模型, 可以在表5中找到。

数据集与基线方法. 我们使用两个知名数据集, LONGMEMEVAL (Wu 等人, 2025) (特别是 LongMemEval-S 划分) 和 Locomo (Maharana 等人, 2024) 来评估记忆能力。我们将 LightMem 与几个对话记忆建模的代表性基线进行比较。1 全文, 2 朴素RAG, 3 LangMem (LangChain, 2025), 4 A-MEM (Xu 等人, 2025), 5 MemoryOS (Kang 等人, 2025), 6 Mem0 (Chhikara 等人, 2025)。此外, 所有方法都使用 GPT-4o-mini、Qwen3-30B-A3B-Instruct-2507 和 GLM-4.6 作为 LLM 主干。关于数据集、基线和实验设置的详细信息, 请参见附录 D。

指标. 我们使用有效性和效率指标评估这些方法。对于有效性, 我们报告 **准确率 (ACC)**, 定义为正确回答问题的比例。评估使用 *GPT-4o-mini* 作为大语言模型裁判, 并由详细的评估提示 (见附录E.1) 指导。对于效率, 我们关注内存库构建阶段 (见第 2.1 节), 所有指标在整个数据集上平均, 因为它与内存系统的设计和实现差异相关。检索和使用阶段不是我们的重点, 因为为了公平比较, The $f_{\text{retrieve}}()$, $f_{\text{chat}}()$ 和检索条目数量在所有方法中相同。因此, 它们的成本只有微小差异, 这个阶段与内存系统的设计基本无关, 如表所示。在内存库构建阶段, 只有两个子过程**摘要**和**更新**涉及使用大语言模型, $f_{\text{sum/extract}}()$ 和 $f_{\text{update}}()$ 。因此, 对于这两个过程, 我们报告大语言模型调用的token消耗, 包括输入 token、输出 token 和总 token 使用量 (以千为单位)。此外, 我们跟踪 **API调用次数**, 统计大语言模型调用的总数, 以及 **运行时间**, 记录内存库构建阶段的总体执行时间。

5.2 主要结果

如表2和表3所示, **LightMem** 在GPT和Qwen两种主干网络上, 对两个数据集都展现出更优越的效果和效率。为了公平比较, 以下分析中LightMem的所有效率指标均指**在线和离线**成本之和。

LongMemEval. 在LongMemEval基准测试中, LightMem在ACC指标上始终优于最强的基线 A-MEM, 使用GPT时准确率提升了2.09%–6.40%,

Table 2: Effectiveness and efficiency comparison on LONGMEMEVAL-S. The token usage is in thousands. – indicates no value for the metric. **Bold** denotes the best result, underline the second-best. r denotes the compression rate. th denotes the capacity threshold of the STM buffer, measured in tokens. Each pair of r and th corresponds to two rows: one for online soft update and one for offline update. OP-update denotes the offline parallel update process of **LightMem**.

Method	ACC (%)	Summary Tokens (k)		Update Tokens (k)		Total (k)	Calls	Runtime (s)
		In	Out	In	Out			
🌀 GPT-4o-mini								
FullText	56.80	–	–	–	–	105.07	–	–
NaiveRAG	61.00	–	–	–	–	–	–	867.38
LangMem	37.20	–	–	982.68	119.48	1,102.16	520.62	2,293.70
A-MEM	62.60	214.66	42.82	1,157.52	190.81	1,605.81	986.55	5,132.06
MemoryOS	44.80	2,302.35	304.18	350.02	35.19	2,991.75	2,938.41	8,030.04
Mem0	53.61	424.13	17.76	560.17	150.56	1,152.62	811.57	4,248.49
LightMem								
$r=0.5, th=256$	64.29	<u>20.80</u>	<u>10.01</u>	–	–	<u>30.81</u>	<u>25.67</u>	<u>302.69</u>
(OP-update)	64.69	–	–	44.46	2.56	47.02	70.23	342.63
$r=0.6, th=256$	<u>67.78</u>	24.58	10.53	–	–	35.11	30.47	329.61
(OP-update)	65.39	–	–	<u>53.98</u>	<u>3.18</u>	57.16	85.07	411.56
$r=0.7, th=512$	68.64	18.88	9.37	–	–	28.25	18.43	283.76
(OP-update)	67.07	–	–	79.38	4.06	83.44	125.47	496.03
🌀 Qwen3-30B-A3B-Instruct-2507								
FullText	54.80	–	–	–	–	105.07	–	–
NaiveRAG	60.80	–	–	–	–	–	–	659.09
LangMem	50.80	–	–	1,311.96	118.06	1,430.02	495.12	3,237.16
A-MEM	65.20	219.21	66.98	1,260.54	318.20	1,864.93	989.30	5,367.51
MemoryOS	49.60	2,101.54	510.88	305.12	27.43	2,944.97	2,922.28	8,721.78
Mem0	39.51	424.20	15.34	411.50	111.35	1001.90	722.76	2,239.94
LightMem								
$r=0.4, th=768$	61.95	9.01	<u>16.14</u>	–	–	25.15	<u>16.54</u>	<u>357.13</u>
(OP-update)	62.34	–	–	111.13	7.88	119.01	176.02	1036.47
$r=0.6, th=768$	70.20	<u>13.19</u>	19.21	–	–	<u>32.40</u>	19.97	417.13
(OP-update)	65.14	–	–	97.11	5.92	103.03	152.93	1023.56
$r=0.8, th=1024$	<u>68.69</u>	14.82	18.49	–	–	33.31	9.43	355.71
(OP-update)	67.34	–	–	<u>106.91</u>	<u>6.20</u>	113.11	168.37	1026.90
📌 GLM-4.6								
FullText	36.71	–	–	–	–	103.38	–	–
NaiveRAG	73.20	–	–	–	–	–	–	53,725.15
LangMem	49.20	–	–	3,052.42	7.03	3,059.45	314.61	5,577.91
A-MEM	70.60	444.95	63.40	1,992.04	403.39	2,903.78	450.40	8,068.80
LightMem								
$r=0.5, th=256$	<u>73.00</u>	<u>16.55</u>	14.06	–	–	<u>30.61</u>	<u>10.78</u>	1,014.37
$r=0.6, th=256$	73.20	16.55	<u>13.99</u>	–	–	<u>30.54</u>	10.78	1,077.69
$r=0.7, th=512$	72.80	16.55	13.91	–	–	30.46	10.78	<u>1,038.19</u>

up to 7.67% with Qwen. In terms of efficiency, for GPT, LightMem reduces total token consumption by $10\times\text{--}38\times$ and API calls by $3.6\times\text{--}30\times$; for Qwen, it reduces total tokens by $6.9\times\text{--}21.8\times$ and API calls by $3.3\times\text{--}17.1\times$. Regarding runtime, LightMem achieves $2.9\times\text{--}12.4\times$ for GPT and $1.6\times\text{--}6.3\times$ for Qwen speedup over other memory baselines.

If considering only online test-time cost, LightMem shows an even larger efficiency advantage. For GPT, LightMem reduces total token consumption by $31.4\times\text{--}105.9\times$ and API calls by $17.1\times\text{--}159.4\times$; for Qwen, it reduces total tokens by $30.1\times\text{--}117.1\times$ and API calls by $24.8\times\text{--}309.9\times$.

LoCoMo. On the LoCoMo dataset, LightMem also demonstrates superior performance over other memory baselines. For the GPT backbone, it improves ACC by 6.10%–18.12%, achieves a $2.87\times\text{--}$

表2：LONGMEMEVAL-S上的有效性和效率对比。token使用以千为单位。–表示该指标无值。**加粗**表示最佳结果，下划线表示次优结果。 r 表示压缩率。 th 表示STM缓冲区的容量阈值，以token衡量。每对 r 和 th 对应两行：一行用于在线软更新，一行用于离线更新。OP更新表示**LightMem**的离线并行更新过程。

方法	准确率 (%)	摘要Token(k)		更新Token (k)		总计(k)		调用次数	时间 (秒)
		In	Out	In	Out	In	Out		
🌀 GPT-4o-mini									
全文	56.80	—	—	—	—	105.07	—	—	
NaiveRAG	61.00	—	—	—	—	—	—	—	867.38
LangMem	37.20	—	—	982.68	119.48	1,102.16	520.62	2,293.70	
A-MEM	62.60	214.66	42.82	1,157.52	190.81	1,605.81	986.55	5,132.06	
MemoryOS	44.80	2,302.35	304.18	350.02	35.19	2,991.75	2,938.41	8,030.04	
Mem0	53.61	424.13	17.76	560.17	150.56	1,152.62	811.57	4,248.49	
LightMem									
$r=0.5, th=256$	64.29	<u>20.80</u>	<u>10.01</u>	—	—	<u>30.81</u>	<u>25.67</u>	<u>302.69</u>	
(OP更新)	64.69	—	—	44.46	2.56	47.02	70.23	342.63	
$r=0.6, th=256$	<u>67.78</u>	24.58	10.53	—	—	35.11	30.47	329.61	
(OP更新)	65.39	—	—	<u>53.98</u>	<u>3.18</u>	57.16	85.07	411.56	
$r=0.7, th=512$	68.64	18.88	9.37	—	—	28.25	18.43	283.76	
(OP更新)	67.07	—	—	79.38	4.06	83.44	125.47	496.03	
🌀 Qwen3-30B-A3B-Instruct-2507									
全文	54.80	—	—	—	—	105.07	—	—	
NaiveRAG	60.80	—	—	—	—	—	—	—	659.09
LangMem	50.80	—	—	1,311.96	118.06	1,430.02	495.12	3,237.16	
A-MEM	65.20	219.21	66.98	1,260.54	318.20	1,864.93	989.30	5,367.51	
MemoryOS	49.60	2,101.54	510.88	305.12	27.43	2,944.97	2,922.28	8,721.78	
Mem0	39.51	424.20	15.34	411.50	111.35	1001.90	722.76	2,239.94	
LightMem									
$r=0.4, th=768$	61.95	9.01	<u>16.14</u>	—	—	25.15	<u>16.54</u>	<u>357.13</u>	
(OP更新)	62.34	—	—	111.13	7.88	119.01	176.02	1036.47	
$r=0.6, th=768$	70.20	<u>13.19</u>	19.21	—	—	<u>32.40</u>	19.97	417.13	
(OP更新)	65.14	—	—	97.11	5.92	103.03	152.93	1023.56	
$r=0.8, th=1024$	<u>68.69</u>	14.82	18.49	—	—	33.31	9.43	355.71	
(OP更新)	67.34	—	—	<u>106.91</u>	<u>6.20</u>	113.11	168.37	1026.90	
📌 GLM-4.6									
全文	36.71	—	—	—	—	103.38	—	—	
NaiveRAG	73.20	—	—	—	—	—	—	—	53,725.15
LangMem	49.20	—	—	3,052.42	7.03	3,059.45	314.61	5,577.91	
A-MEM	70.60	444.95	63.40	1,992.04	403.39	2,903.78	450.40	8,068.80	
LightMem									
$r=0.5, th=256$	73.00	<u>16.55</u>	14.06	—	—	<u>30.61</u>	<u>10.78</u>	1,014.37	
$r=0.6, th=256$	73.20	16.55	<u>13.99</u>	—	—	<u>30.54</u>	10.78	1,077.69	
$r=0.7, th=512$	72.80	16.55	13.91	—	—	30.46	10.78	<u>1,038.19</u>	

使用Qwen时则提升了高达7.67%。在效率方面，对于GPT，LightMem将总token消耗减少了 $10\times\text{--}38\times$ ，API调用次数减少了 $3.6\times\text{--}30\times$ ；对于Qwen，它将总token消耗减少了 $6.9\times\text{--}21.8\times$ ，API调用次数减少了 $3.3\times\text{--}17.1\times$ 。在运行时方面，LightMem相较于其他内存基线，对GPT实现了 $2.9\times\text{--}12.4\times$ 的速度提升，对Qwen实现了 $1.6\times\text{--}6.3\times$ 的速度提升。

如果仅考虑在线测试时成本，LightMem 显示出更大的效率优势。对于 GPT，LightMem 减少了总 token 消耗 $31.4\times\text{--}105.9\times$ 和 API 调用 $17.1\times\text{--}159.4\times$ ；对于 Qwen，它减少了总 token 消耗 $30.1\times\text{--}117.1\times$ 和 API 调用 $24.8\times\text{--}309.9\times$ 。

LoCoMo. 在LoCoMo数据集上，LightMem也表现出优于其他内存基线的性能。对于GPT主干网络，它将ACC提升了6.10%–18.12%，实现了 $2.87\times\text{--}$

Table 3: Effectiveness and efficiency comparison on LoCoMo. Due to space limitations and for ease of comparison, we merge the results before and after LightMem’s offline update into a single row. The ACC reported corresponds to the performance after the offline update.

Method	ACC (%)	Summary Tokens (k)		Update Tokens (k)		Total (k)	Calls	Runtime (s)
		In	Out	In	Out			
🌀 GPT-4o-mini								
FullText	71.83	–	–	–	–	–	–	–
NaiveRAG	63.64	–	–	–	–	–	–	–
LangMem	57.20	–	–	898.27	111.95	1010.22	920.62	2229.37
A-MEM	64.16	182.74	49.29	729.89	187.52	1149.43	1175.47	6060.73
MemoryOS(locomo) ¹	58.25	110.98	33.40	78.08	64.54	287.00	553.45	2422.05
MemoryOS(regular)	54.87	226.86	46.61	177.66	75.34	526.48	1016.06	3332.59
Mem0	61.69	851.32	20.53	632.12	189.42	1693.39	1602.20	4432.87
LightMem(0.7,512)	<u>71.95</u>	<u>73.19</u>	<u>20.13</u>	<u>6.05</u>	<u>0.40</u>	<u>99.76</u>	<u>41.65</u>	<u>848.49</u>
LightMem(0.7,768)	70.26	57.54	<u>18.92</u>	3.79	0.23	80.48	29.55	737.80
LightMem(0.8,768)	72.99	<u>62.82</u>	17.95	<u>4.14</u>	<u>0.28</u>	<u>85.19</u>	<u>29.83</u>	<u>815.32</u>
🚀 Qwen3-30B-A3B-Instruct-2507								
FullText	74.87	–	–	–	–	–	–	–
NaiveRAG	66.95	–	–	–	–	–	–	–
LangMem	60.53	–	–	1004.35	138.02	1142.37	1005.37	2268.57
A-MEM	56.10	158.29	60.85	924.19	483.51	1626.80	1175.40	5543.90
MemoryOS(locomo)	61.04	122.21	53.12	104.43	81.75	361.51	414.70	1269.70
MemoryOS(regular)	51.30	228.85	51.60	242.27	143.63	666.35	1004.60	1982.20
Mem0	43.31	827.09	18.64	763.88	189.80	1799.40	1614.50	4540.70
LightMem(0.6,768)	71.36	56.68	<u>34.14</u>	8.31	0.74	99.87	29.10	815.70
LightMem(0.8,1024)	<u>72.60</u>	<u>61.38</u>	36.33	<u>9.86</u>	<u>0.88</u>	<u>108.45</u>	<u>32.00</u>	<u>1079.40</u>

$20.92\times$ improvement in total token efficiency, reduces API calls by $13.29\times$ – $39.78\times$, and accelerates runtime by $2.63\times$ – $8.21\times$. On the Qwen backbone, LightMem maintains its advantage in both effectiveness and efficiency, with 4.41%–29.29% higher ACC, $3.33\times$ – $18.02\times$ reduction in total token consumption, $12.96\times$ – $55.48\times$ fewer API calls, and $1.18\times$ – $5.57\times$ faster runtime.

LightMem achieves superior performance on nearly all metrics and both LLM backbones, while demonstrating robust performance and efficiency on both LongMemEval and LoCoMo, highlighting its generalizability across different models and scenarios.

5.3 ANALYSIS OF PRE-COMPRESSING SUBMODULE

Performance and Overhead. LightMem uses an additional model (Pan et al., 2024b; Xia et al., 2025) for pre-compression. We evaluate its performance by randomly sampling 1/5 of LONG-MEMEval and compressing it at ratios shown in Figure 3(a), then prompting LLMs for in-context QA. When compression ratio r ranges from 50%–80%, compressed and uncompressed performance are comparable, demonstrating LLMs can effectively understand compressed content and validating LightMem’s approach. The submodule is highly efficient, consuming under 2GB of GPU memory with negligible impact on overall runtime.

Impact of r on Performance. As shown in Tables 8 and 9, The optimal r for ACC is dependent on the STM buffer threshold th . For smaller thresholds ($th \in \{0, 256\}$), an r of 0.6 achieves the highest ACC. In contrast, for larger thresholds ($th \in \{512, 1024\}$), a higher retention rate of $r = 0.7$ performs best. This suggests greater buffer capacity enables effective use of richer, less-compressed information, leveraging LLMs’ advanced long-context processing to mitigate the “lost in the middle” phenomenon. On average, the optimal r for ACC is 0.6, reflecting a trade-off between information compression rate and the quantity of information in the STM buffer. In terms of efficiency, a lower r

¹MemoryOS(locomo) is the LoCoMo reproduction script in the MemoryOS library, simplifying the standard version, shown as MemoryOS(regular).

表 3：在 LOCOMO 上的有效性和效率对比。由于篇幅限制且便于比较，我们将 LightMem 离线更新前后的结果合并为一行。所报告的 ACC 对应于离线更新后的性能。

方法	准确率 (%)	摘要Token(k)		更新Token (k)		总计(k)		调用次数	时间 (秒)
		In	Out	In	Out	In	Out		
🌀 GPT-4o-mini									
全文	71.83	—	—	—	—	—	—	—	—
NaiveRAG	63.64	—	—	—	—	—	—	—	—
LangMem	57.20	—	—	898.27	111.95	1010.22	920.62	2229.37	2229.37
A-MEM	64.16	182.74	49.29	729.89	187.52	1149.43	1	6060.73	6060.73
MemoryOS(LoCoMo) ¹	58.25	110.98	33.40	78.08	64.54	287.00	553.45	2422.05	2422.05
MemoryOS(regular)	54.87	226.86	46.61	177.66	75.34	526.48	1016.06	3332.59	3332.59
Mem0	61.69	851.32	20.53	632.12	189.42	1693.39	1	4432.87	4432.87
LightMem(0.7,512)	<u>71.95</u>	<u>73.19</u>	<u>20.13</u>	<u>6.05</u>	<u>0.40</u>	<u>99.76</u>	<u>41.65</u>	<u>848.49</u>	<u>848.49</u>
LightMem(0.7,768)	70.26	57.54	<u>18.92</u>	3.79	0.23	80.48	29.55	737.80	737.80
LightMem(0.8,768)	72.99	<u>62.82</u>	17.95	<u>4.14</u>	<u>0.28</u>	<u>85.19</u>	<u>29.83</u>	<u>815.32</u>	<u>815.32</u>
🚀 Qwen3-30B-A3B-Instruct-2507									
全文	74.87	—	—	—	—	—	—	—	—
NaiveRAG	66.95	—	—	—	—	—	—	—	—
LangMem	60.53	—	—	1004.35	138.02	1142.37	1	2268.57	2268.57
A-MEM	56.10	158.29	60.85	924.19	483.51	1626.80	1	5543.90	5543.90
MemoryOS(LoCoMo)	61.04	122.21	53.12	104.43	81.75	361.51	414.70	1269.70	1269.70
MemoryOS(regular)	51.30	228.85	51.60	242.27	143.63	666.35	1004.60	1982.20	1982.20
Mem0	43.31	827.09	18.64	763.88	189.80	1799.40	1	4540.70	4540.70
LightMem(0.6,768)	71.36	56.68	<u>34.14</u>	8.31	0.74	99.87	29.10	815.70	815.70
LightMem(0.8,1024)	<u>72.60</u>	<u>61.38</u>	36.33	<u>9.86</u>	<u>0.88</u>	<u>108.45</u>	<u>32.00</u>	<u>1079.40</u>	<u>1079.40</u>

$20.92\times$ 在总token效率上的提升，将API调用次数减少了 $13.29\times$ – $39.78\times$ ，并将运行时加速了 $2.63\times$ – $8.21\times$ 。在Qwen主干网络上，LightMem在有效性和效率两方面都保持着优势，ACC提升了4.41%–29.29%，总token消耗减少了 $3.33\times$ – $18.02\times$ ，API调用次数减少了 $12.96\times$ – $55.48\times$ ，运行时加速了 $1.18\times$ – $5.57\times$ 。

LightMem 在几乎所有指标和两种 LLM 主干上均实现了优越的性能，同时在 LongMemEval 和 LoCoMo 上展现了稳健的性能和效率，突显了其在不同模型和场景下的泛化能力。

5.3 预压缩子模块分析

性能与开销。 LightMem 使用一个额外的模型（Pan 等人.,2024b; Xia 等人.,2025）进行预压缩。我们通过随机采样 LONG-MEMEval 的 1/5 并按照图 3(a) 中所示的比例进行压缩，然后提示大语言模型进行情境问答。当压缩率 在 50%–80% 之间时，压缩与未压缩的性能相当，这表明大语言模型能够有效地理解压缩内容，并验证了 LightMem 的方法。该子模块非常高效，占用不到 2GB 的 GPU 内存，且对整体运行时间的影响可以忽略不计。

对性能的影响。 r 。如表8 和 9所示，ACC的最优 r 取决于STM缓冲区阈值 th 。对于较小的阈值（ $th \in \{0, 256\}$ ），一个 r 为0.6的 $r = 0.7$ 能实现最高的ACC。相比之下，对于较大的阈值（ $th \in \{512, 1024\}$ ），更高的 $r = 0.7$ 表现最佳。这表明更大的缓冲区容量能够有效利用更丰富、压缩率较低的信息，利用大语言模型先进的长上下文处理能力来缓解“中间丢失”现象。平均而言，ACC的最优 r 为0.6，反映了信息压缩率与STM缓冲区中信息数量之间的权衡。在效率方面，较低的 r

¹MemoryOS(locomo)是MemoryOS库中的LoCoMo复现脚本，简化了标准版本，表现为MemoryOS(regular)。

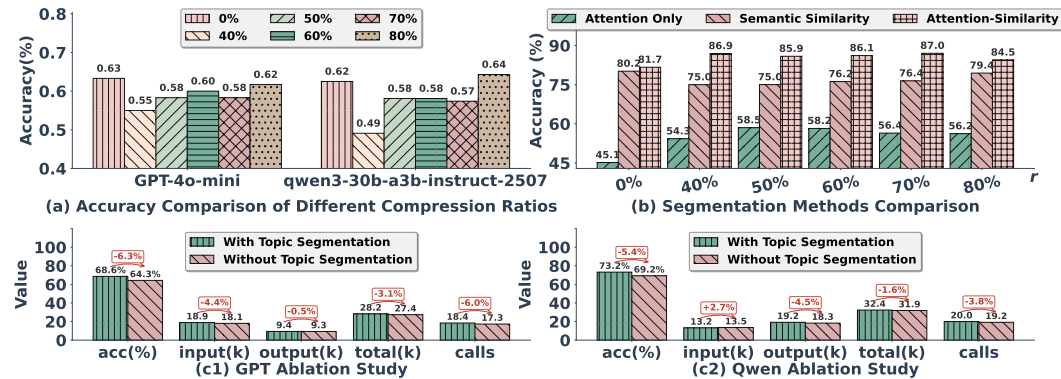


Figure 3: Analysis and Ablation Study of Key Modules. Fig.(a) depicts the QA accuracy when using prompts compressed at different ratios (r) as in-contexts to query the LLM directly. Fig.(b) compares the accuracy of different topic segmentation methods under these varying compression ratios. Fig.(c1) and Fig.(c2) present the ablation study for the topic segmentation module, evaluating its impact on both performance and efficiency for the GPT and Qwen models.

generally leads to higher efficiency, as it triggers the buffer threshold less frequently under the same th , resulting in fewer API calls and lower token consumption.

5.4 ANALYSIS OF TOPIC SEGMENTATION SUBMODULE

Segmentation Accuracy. To validate the accuracy of our proposed hybrid topic segmentation method, we compare it with segmentation using only a single granularity: attention-only-based and similarity-only-based segmentation. Since the construction process of the LONGMEMEVAL indicates that different sessions naturally serve as topic boundaries, we directly use them as ground-truth labels. The final accuracy is calculated as the number of correctly identified segmentation points divided by the total number of labels. The results in Figure 3(b) validate the effectiveness of our method: it achieves higher accuracy than both individual segmentation methods across all compression ratios, with an absolute accuracy exceeding 80%.

Ablation Study. As shown in Figure 3(c), removing the topic segmentation submodule slightly improves efficiency but significantly harms accuracy, causing a 6.3% drop for GPT and 5.4% for Qwen. This indicates that the submodule effectively enables models to perceive semantic units in the input, facilitating subsequent memory unit generation.

5.5 ANALYSIS OF THE STM THRESHOLD’S IMPACT

As illustrated in the Figure 4, the STM buffer threshold (th) has a distinct but significant impact on both efficiency and performance metrics. A consistent trend is: as th increases, there is a marked improvement in efficiency. In contrast, the effect on QA accuracy is non-monotonic. The optimal threshold for accuracy varies depending on the model and the compression ratio (r), indicating that a larger buffer does not always yield better performance. This highlights a crucial trade-off: while a larger STM threshold is consistently better for reducing computational cost, the ideal setting for maximizing task accuracy requires careful tuning.

5.6 ANALYSIS OF SLEEP-TIME UPDATE

Why Soft Updates Work. A primary challenge in designing memory systems is handling updates. While powerful, LLMs can be unreliable when tasked with complex real-time update operations. For instance, when presented with two related but not contradictory pieces of information, an LLM might incorrectly interpret them as a conflict and delete the older memory entry, leading to irreversible information loss. Instead, the optimal operations might be to merge the information or

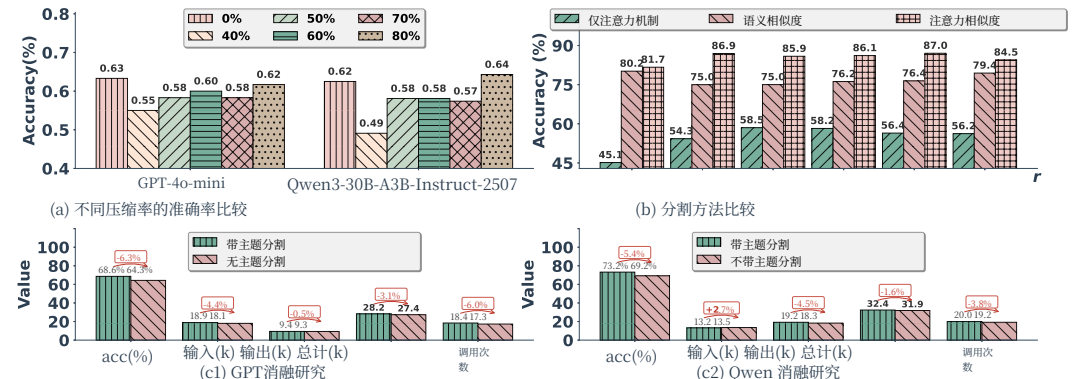


图3: 关键模块的分析与消融研究。图(a)描绘了使用不同压缩率(r)作为上下文提示直接查询大语言模型的问答准确率。图(b)比较了在上述不同压缩率下, 不同主题分割方法的准确率。图(c1)和图(c2)展示了主题分割模块的消融研究, 评估其对GPT和Qwen模型的性能与效率的影响。

通常会导致更高的效率, 因为它在相同的 th 下触发缓冲区阈值更少, 从而导致API调用次数更少和token消耗更低。

5.4 主题分割子模块分析

分割准确率。 为验证我们提出的混合主题分割方法的准确性, 我们将其与仅使用单一粒度的分割方法进行比较: 仅基于注意力分割和仅基于相似度分割。由于 LONGMEMEVALALUSETION表明不同会话自然作为主题边界, 我们直接将其用作真实标签。最终准确率计算为正确识别的分割点数量除以总标签数量。图3(b)的结果验证了我们的方法的有效性: 它在所有压缩率下都实现了比两种单一分割方法更高的准确率, 绝对准确率超过80%。

消融研究。 如图 3(c) 所示, 移除主题分割子模块略微提高了效率, 但显著降低了准确率, 导致 GPT 下降 6.3%, Qwen 下降 5.4%。这表明该子模块有效地使模型能够感知输入中的语义单元, 从而促进后续记忆单元的生成。

5.5 STM 阈值的影响分析

如图 4所示, STM 缓冲区阈值 (th) 对效率和性能指标均有显著但不同的影响。一个一致的趋势是: 随着 th 的增加, 效率显著提高。相比之下, 对问答准确率的影响是非单调的。准确率的最优阈值取决于模型和压缩率 (r), 表明更大的缓冲区并不总是带来更好的性能。这突显了一个关键的权衡: 虽然更大的 STM 阈值始终有助于降低计算成本, 但最大化任务准确率的最优设置需要仔细调优。

5.6 睡眠时间更新分析

为什么软更新有效。 设计内存系统面临的主要挑战之一是处理更新。虽然强大, 但大语言模型在执行复杂的实时更新操作时可能不可靠。例如, 当向大语言模型呈现两段相关但并不矛盾的信息时, 它可能会错误地将其解释为冲突并删除较旧的内存条目, 导致不可逆的信息丢失。相反, 最佳操作可能是合并信息或

simply add the new entry. In contrast, **LightMem** performs only incremental additions through soft updates during test time, which preserves global information and complete semantics.

Case Study: Memory Update Mechanism Comparison

History1: {'Monday, 2 PM': User is planning a trip to Tokyo.}	
History2: {'Monday, 4 PM': User asks about trains to Kyoto.}	
Hard Update: Overwrites memory	LightMem Soft Update: Appends info
-> "User plans Kyoto trip"	-> "Tokyo trip + Kyoto inquiry"
⚠️ Tokyo context lost	✅ Full context preserved

6 RELATED WORK

Hard Prompt Compression for LLMs. Hard prompt compression improves LLM efficiency by removing redundant content from prompts (Li et al., 2025c). Methods recently have evolved from using smaller language models (Jiang et al., 2023; Li et al., 2023; Chuang et al., 2024) to query-aware approaches that preserve task-relevant information (Weston & Sukhbaatar, 2023; Creswell et al., 2023; Jiang et al., 2024). Additionally, lightweight bidirectional encoders have demonstrated strong effectiveness and efficiency (Pan et al., 2024a; Liskavets et al., 2025).

Chunking Strategies in RAG Systems. Retrieval-Augmented Generation (RAG) systems rely on chunking external documents into smaller units for retrieval (Lewis et al., 2020; Gao et al., 2023). Existing chunking strategies include rule-based methods creating fixed-size segments (Lewis et al., 2020; Sarthi et al., 2024; Edge et al., 2024; Gutierrez et al., 2024), semantic-based methods grouping content by topic (Qu et al., 2025), and LLM-driven methods leveraging model knowledge for splitting (Pan et al., 2025; Duarte et al., 2024; Zhao et al., 2024; Liu et al., 2025b). However, all of these chunking strategies for RAG systems are tailored to static scenarios, not applicable to dynamic and open-ended environments.

Memory Systems for LLM Agents. Memory systems help LLM agents move beyond stateless interactions to support flexible reasoning and adaptation in complex and changing environments (Liu et al., 2025a; Mei et al., 2025). The earliest and most straightforward approaches store experiences as linear or sequential streams, sometimes enhanced with hierarchical structures (Liang et al., 2023; Park et al., 2023; Packer et al., 2023; Zhong et al., 2024; Salama et al., 2025; Fang et al., 2025). A more structured class of methods represents memories as nodes and their relationships as edges, using trees, graphs, or temporal knowledge structures to support retrieval and update (Rezazadeh et al., 2025; Chhikara et al., 2025; Rasmussen et al., 2025; Xu et al., 2025; Zhang et al., 2025). The latest trend integrates various types of memory, allowing them to interact and synergistically improve overall performance (Kang et al., 2025; Li et al., 2025b; Wang & Chen, 2025; Nan et al., 2025). Overall, existing memory systems for LLM agents have become increasingly complex and capable, leveraging hierarchical, structured, and multi-type memories. However, most focus on maximizing effectiveness, with limited consideration of efficiency. While some recent works (Guo et al., 2024; Zhao et al., 2025; Dong et al., 2025) share a similar motivation with our work, they focus on lightweight adaptations of GraphRAG where the corpus is predefined and static.

7 CONCLUSION

In this work, we introduced LightMem, a lightweight and efficient memory framework designed to address the significant overhead of memory systems for LLM agents. Inspired by the multi-stage Atkinson-Shiffrin human memory model, LightMem’s architecture effectively filters, organizes, and consolidates information. Our empirical evaluation demonstrates that this approach maintains strong task performance while sharply reducing computational costs. In the near future, we plan to accelerate LightMem’s update phase via offline pre-computed KV caches, reducing runtime overhead. We aim to integrate a lightweight knowledge graph memory for explicit multi-hop reasoning and structured retrieval. A multimodal memory extension will enable adaptation to visual, auditory, and textual inputs in embodied and real-world scenarios.

直接添加新条目。相比之下， **LightMem** 在测试期间仅通过软更新进行增量添加，从而保留了全局信息和完整语义。

案例研究：内存更新机制对比

History1: { ‘周一，下午2点’：用户计划前往东京旅行。}	History2: { ‘周一，下午4点’：用户询问前往京都的火车信息。}
硬更新: 覆盖内存	LightMem软更新: 附加信息
-> "用户计划京都旅行"	-> "东京之行 + 京都查询"
⚠️ 东京上下文丢失	✅ 全文保留

6相关工作

大语言模型的硬提示压缩。硬提示压缩通过从提示中移除冗余内容来提高大语言模型的效率（Li等人，2025c）。最近的方法已从使用更小的语言模型（Jiang等人，2023；Li等人，2023；Chuang等人，2024）发展到查询感知方法，这些方法保留任务相关信息（Weston & Sukhbaatar, 2023; Creswell等人，2023; Jiang等人，2024）。此外，轻量级双向编码器已展现出强大的有效性和效率（Pan等人，2024a; Liskavets等人，2025）。

RAG系统中的分块策略。检索增强生成（RAG）系统依赖于将外部文档分块为更小的单元以供检索（Lewis等人，2020; Gao等人，2023）。现有的分块策略包括基于规则的创建固定大小片段的方法（Lewis等人，2020; Sarthi等人，2024; Edge等人，2024; Gutierrez等人，2024），基于语义的方法按主题分组内容（Qu等人，2025），以及利用模型知识进行分割的大语言模型驱动方法（Pan等人，2025; Duarte等人，2024; Zhao等人，2024; Liu等人，2025b）。然而，所有这些RAG系统的分块策略都是针对静态场景设计的，不适用于动态和开放的环境。

用于大语言模型代理的内存系统。内存系统帮助大语言模型代理超越无状态交互，以支持在复杂和变化的环境中灵活推理和适应（刘等人，2025a; 梅等人，2025）。最早且最直接的方法将经验存储为线性或顺序流，有时会增强层次结构（梁等人，2023; 朴等人，2023; 帕克等人，2023; 钟等人，2024; 萨拉马等人，2025; 方等人，2025）。更结构化的方法将记忆表示为节点，将它们之间的关系表示为边，使用树、图或时间知识结构来支持检索和更新（雷扎扎德等人，2025; Chhikara 等人，2025; 拉斯穆森等人，2025; 徐等人，2025; 张等人，2025）。最新趋势将各种类型的记忆集成起来，允许它们交互并协同提高整体性能（Kang 等人，2025; 李等人，2025b; 王与陈，2025; 南等人，2025）。总的来说，现有的大语言模型代理内存系统变得越来越复杂和强大，利用层次结构、结构化和多类型记忆。然而，大多数关注于最大化有效性，对效率的考虑有限。虽然一些近期工作（郭等人，2024; 赵等人，2025; 董等人，2025）与我们的工作有相似的动机，但它们专注于图RAG的轻量级适应，其中语料库是预定义且静态的。

7 结论

在这项工作中，我们介绍了 LightMem，一个轻量级且高效的内存框架，旨在解决大语言模型代理的内存系统带来的显著开销。受多阶段阿特金森-希夫林人类记忆模型的启发，LightMem 的架构能够有效过滤、组织和整合信息。我们的实证评估表明，这种方法在保持强大任务性能的同时，显著降低了计算成本。在不久的将来，我们计划通过离线预计算的 KV 缓存加速 LightMem 的更新阶段，从而减少运行时开销。我们旨在集成轻量级知识图谱内存，以支持显式的多跳推理和结构化检索。多模态内存扩展将使系统能够适应具身化和现实世界场景中的视觉、听觉和文本输入。

ETHICS STATEMENT

LightMem enhances LLM agents by creating an external memory of user interactions. While this improves agent coherence, it introduces critical ethical challenges. Storing dialogue histories poses inherent risks to user privacy, as conversations may contain sensitive data. The memory can also absorb and perpetuate biases or misinformation from user input, potentially leading to bad agent behavior. Therefore, any deployment of this technology must prioritize robust safeguards. We strongly advocate for strict privacy protocols, such as data anonymization and user consent, as well as mechanisms to mitigate the effects of biased or false memories. Responsible development is essential to ensure these memory-augmented systems are used in a safe and trustworthy manner.

REPRODUCIBILITY STATEMENT

To ensure the reproducibility of this work, we introduce the detailed implementations for LightMem are provided in in Section 3, Appendix C. Additionally, we plan to release our source code in the future to further support reproducibility. These measures are intended to facilitate the verification and replication of our results by other researchers in the field.

ACKNOWLEDGMENTS

We would like to express our sincere gratitude to the anonymous reviewers for their thoughtful and constructive feedback. This work was supported by the National Natural Science Foundation of China (No. 62576307, No. NSFCU23B2055, No. NSFCU19B2027), the Fundamental Research Funds for the Central Universities (226-2023-00138), the 2025 Zhejiang Provincial Center for Disease Control and Prevention Science and Technology Talent Incubation Project (No. 2025-A-04), the 2025 Zhejiang Health Informatics Association Scientific Research Program (Key Project, No. 2025XHZN-Z01), titled “Research on Monitoring and Early Warning Methods of AI Large Model and Infectious Disease Epidemic Data Fusion”, undertaken by the Zhejiang Provincial Center for Disease Control and Prevention, Yongjiang Talent Introduction Programme (2021A-156-G), and Information Technology Center and State Key Lab of CAD&CG, Zhejiang University.

REFERENCES

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023.

Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time. *arXiv preprint arXiv:2501.00663*, 2024.

Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready ai agents with scalable long-term memory. *ArXiv*, abs/2504.19413, 2025. URL <https://api.semanticscholar.org/CorpusID:278165315>.

Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, and Xia Ben Hu. Learning to compress prompt in natural language formats. In Kevin Duh, Helena Gómez-Adorno, and Steven Bethard (eds.), *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, NAACL 2024, Mexico City, Mexico, June 16-21, 2024, pp. 7756–7767. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.NAACL-LONG.429. URL <https://doi.org/10.18653/v1/2024.naacl-long.429>.

Antonia Creswell, Murray Shanahan, and Irina Higgins. Selection-inference: Exploiting large language models for interpretable logical reasoning. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net, 2023. URL <https://openreview.net/forum?id=3Pf3Wg6o-A4>.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shirong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan,

伦理声明

LightMem 通过创建用户交互的外部内存来增强大语言模型代理。虽然这提高了代理的连贯性，但也引入了关键性的伦理挑战。存储对话历史会对用户隐私构成固有风险，因为对话可能包含敏感数据。内存还可能吸收并持续传播用户输入中的偏见或错误信息，可能导致代理行为不当。因此，任何此类技术的部署都必须优先考虑强大的安全措施。我们强烈主张实施严格的隐私协议，例如数据匿名化和用户同意，以及缓解偏见或错误记忆影响的机制。负责任的开发对于确保这些内存增强系统以安全可靠的方式使用至关重要。

可复现性声明

为确保本工作的可复现性，我们提供了LightMem的详细实现，位于第3节附录C。此外，我们计划未来发布源代码以进一步支持可复现性。这些措施旨在促进该领域其他研究人员对结果的验证和复制。

致谢

我们衷心感谢匿名审稿人提出的周到和建设性意见。本研究由国家自然科学基金（编号：62576307、NSFCU23B2055、NSFCU19B2027）、中央高校基本科研业务费专项资金（226-2023-00138）、2025年浙江省疾病预防控制科学技术人才孵化项目（编号：2025-A-04）、2025年浙江省卫生健康协会科学研究项目（重点项目，编号：2025XHZN-Z01），题为“人工智能大模型与传染病疫情数据融合的监测预警方法研究”，由浙江省疾病预防控制中心承担，以及永嘉人才引进计划（2021A-156-G）和浙江大学信息技术中心及计算机辅助设计与图形学国家重点实验室资助。

参考文献

Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, 等. GPT-4 技术报告. *arXiv* 预印本 *arXiv:2303.08774*, 2023. Ali Behrouz, Peilin Zhong, 和 Vahab Mirrokni. Titans: 在测试时学习记忆. *arXiv* 预印本 *arXiv:2501.00663*, 2024. Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, 和 Deshraj Yadav. Mem0: 构建具有可扩展长期记忆的生产就绪 AI 代理. *ArXiv*, abs/2504.19413, 2025. URL<https://api.semanticscholar.org/CorpusID:278165315>. Yu-Neng Chuang, Tianwei Xing, Chia-Yuan Chang, Zirui Liu, Xun Chen, 和 Xia Ben Hu. 学习在自然语言格式中压缩提示. 在 Kevin Duh, Helena Gómez-Adorno, 和 Steven Bethard (eds.), 2024 年北美计算语言学协会分会会议：人机语言技术 (第一卷：长文), *NAACL 2024*, 墨西哥城, 墨西哥, 2024 年 6 月 16-21 日, 第 7756-7767 页. 计算语言学协会, 2024. doi: 10.18653/V1/2024.NAACL-LONG.429. URL<https://doi.org/10.18653/v1/2024.naacl-long.429>. Antonia Creswell, Murray Shanahan, 和 Irina Higgins. 选择推理：利用大型语言模型进行可解释的逻辑推理. 在 第 11 届国际学习表示会议, *ICLR 2023*, 基加利, 卢旺达, 2023 年 5 月 1-5 日. OpenReview.net, 2023.URL <https://openreview.net/forum?id=3Pf3Wg6o-A4>.

DeepSeek-AI, 戴亚谷, 杨德建, 张浩伟, 宋俊晓, 张若宇, 徐润欣, 朱启浩, 马世荣, 王培怡, 毕晓, 张晓康, 余兴凯, 吴宇, 吴志峰, 勾志斌, 邵志宏, 李卓舒, 高子毅, 刘爱金, 薛冰, 王冰轩, 吴博文, 冯北, 陆成达, 赵成刚, 邓成奇, 张晨宇, 阮崇,

Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiushi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, and S. S. Li. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *CoRR*, abs/2501.12948, 2025. doi: 10.48550/ARXIV.2501.12948. URL <https://doi.org/10.48550/arXiv.2501.12948>.

Junnan Dong, Siyu An, Yifei Yu, Qian-Wen Zhang, Linhao Luo, Xiao Huang, Yunsheng Wu, Di Yin, and Xing Sun. Youtu-graphrag: Vertically unified agents for graph retrieval-augmented complex reasoning. *arXiv preprint arXiv:2508.19855*, 2025.

Yiming Du, Wenyu Huang, Danna Zheng, Zhaowei Wang, Sebastien Montella, Mirella Lapata, Kam-Fai Wong, and Jeff Z Pan. Rethinking memory in ai: Taxonomy, operations, topics, and future directions. *arXiv preprint arXiv:2505.00675*, 2025.

André V. Duarte, João Marques, Miguel Graça, Miguel Freire, Lei Li, and Arlindo L. Oliveira. Lumberchunker: Long-form narrative document segmentation. In Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2024, Miami, Florida, USA, November 12-16, 2024*, pp. 6473–6486. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.FINDINGS-EMNLP.377. URL <https://doi.org/10.18653/v1/2024.findings-emnlp.377>.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, and Jonathan Larson. From local to global: A graph rag approach to query-focused summarization. *ArXiv*, abs/2404.16130, 2024. URL <https://api.semanticscholar.org/CorpusID:269363075>.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, and Ningyu Zhang. Memp: Exploring agent procedural memory, 2025. URL <https://arxiv.org/abs/2508.06433>.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. Lightrag: Simple and fast retrieval-augmented generation. *arXiv preprint arXiv:2410.05779*, 2024.

Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. Hipporag: Neurobiologically inspired long-term memory for large language models. In Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, and Cheng Zhang (eds.), *Advances in Neural Information Processing Systems 38: Annual Conference on Neural Information Processing Systems 2024, NeurIPS 2024, Vancouver, BC, Canada, December 10 - 15, 2024*, 2024. URL http://papers.nips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract-Conference.html.

Yuanzhe Hu, Yu Wang, and Julian McAuley. Evaluating memory in llm agents via incremental multi-turn interactions, 2025. URL <https://arxiv.org/abs/2507.05257>.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Llmlingua: Compressing prompts for accelerated inference of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pp. 13358–13376. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.825. URL <https://doi.org/10.18653/v1/2023.emnlp-main.825>.

戴麦代, 陈德立, 姬东杰, 李二航, 林方云, 戴福丛, 罗福利, 郝光波, 陈冠廷, 李国伟, 张汉, 鲍汉, 徐汉伟, 王浩程, 丁红会, 辛华健, 高华卓, 曲辉, 李辉, 郭建忠, 李嘉石, 王佳伟, 陈景长, 袁景阳, 邱俊杰, 李俊龙, 蔡建良, 李佳琪, 梁建, 陈金, 董凯, 胡凯, 高凯歌, 关康, 黄克欣, 于快, 王 Lean, 张乐丛, 赵亮, 王立通, 张丽月, 徐雷, 夏雷毅, 张明川, 张明华, 唐明辉, 李梦, 王妙军, 李明明, 田宁, 黄盼盼, 张鹏, 王前程, 陈倩雨, 杜秋实, 葛瑞琪, 张瑞松, 潘瑞哲, 王润基, 陈瑞金, 金瑞霖, 陈睿毅, 陆上好, 周上岩, 陈山黄, 叶胜风, 王世宇, 余水萍, 周顺峰, 潘淑婷, 以及李思思. Deepseek-r1: 通过强化学习激励大语言模型的推理能力. *arXiv*, abs/2501.12948, 2025. doi: 10.48550/ARXIV.2501.12948. URL<https://doi.org/10.48550/arXiv.2501.12948>.

董俊南, 安思语, 余一飞, 张千文, 罗林浩, 黄晓, 吴云胜, 隋迪, 孙行. Youtu-graphrag: 用于图检索增强复杂推理的垂直统一智能体. *arXiv预印本 arXiv:2508.19855*, 2025.

杜一鸣, 黄文宇, 郑丹娜, 王兆伟, Montella塞巴斯蒂安, Lapata米雷拉, Wong Kam-Fai, 和 Pan Jeff Z. 重新思考AI中的内存: 分类, 操作, 主题和未来方向. *arXiv预印本 arXiv:2505.00675*, 2025.

André V. Duarte, João Marques, Miguel Graça, Miguel Freire, 李雷, 和 Arlindo L. Oliveira. Lumberchunker: 长文本叙事文档分割. 收录于Yaser Al-Onaizan, Mohit Bansal, 和 Yun-Nung Chen (编), 计算语言学协会发现: *EMNLP 2024*, 佛罗里达州迈阿密, 美国, 2024年11月12-16日, 第6473-6486页. 计算语言学协会, 2024. doi: 10.18653/V1/2024.FINDINGS-EMNLP.377. URL<https://doi.org/10.18653/v1/2024.findings-emnlp.377>.

Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, 和 Jonathan Larson. 从本地到全局: 一种面向查询的图检索摘要方法. *ArXiv*, abs/2404.16130, 2024. URL<https://api.semanticscholar.org/CorpusID:269363075>.

Runnan Fang, Yuan Liang, Xiaobin Wang, Jialong Wu, Shuofei Qiao, Pengjun Xie, Fei Huang, Huajun Chen, 和 Ningyu Zhang. Memp: 探索智能体程序记忆, 2025. URL<https://arxiv.org/abs/2508.06433>.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, 和 Haofen Wang. 面向大型语言模型的检索增强生成: 一项调查. *arXiv预印本 arXiv:2312.10997*, 2023.

郭子睿, 夏良浩, 余彦华, 阿奥, 和 黄超. Lightrag: 简单且快速的检索增强生成. *arXiv预印本 arXiv:2410.05779*, 2024.

Bernal Jimenez Gutierrez, 舒一恒, 余宇, 雅木浩也, 和 苏宇. Hipporag: 受神经生物学启发的长期记忆用于大型语言模型. 在 Amir Globersons, Lester Mackey, Danielle Belgrave, Angela Fan, Ulrich Paquet, Jakub M. Tomczak, 和 张成 (eds.), 神经信息处理系统进展 38: 2024 年神经信息处理系统大会, *NeurIPS 2024*, 加拿大温哥华, BC, 2024 年 12 月 10 - 15 日, 2024. URLhttp://papers.nips.cc/paper_files/paper/2024/hash/6ddc001d07ca4f319af96a3024f6dbd1-Abstract-Conference.html.

袁哲, 王宇, 和朱利安·麦克劳利. 通过增量多回合交互评估大语言模型代理中的记忆, 2025年. URL<https://arxiv.org/abs/2507.05257>.

蒋辉强, 吴乾辉, 林钦耀, 杨宇清, 邱丽丽. Llmlingua: 压缩提示以加速大型语言模型的推理. 收录于侯达·布阿莫尔、胡安·皮诺和卡拉卡·巴利 (编) 的《2023年自然语言处理经验方法会议论文集, EMNLP 2023》, 新加坡, 2023年12月6日至10日, 第13358-13376页. 计算语言学协会, 2023年. doi: 10.18653/V1/2023.EMNLP-MAIN.825. URL<https://doi.org/10.18653/v1/2023.emnlp-main.825>.

Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. Longllmlingua: Accelerating and enhancing llms in long context scenarios via prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, ACL 2024, Bangkok, Thailand, August 11-16, 2024, pp. 1658–1677. Association for Computational Linguistics, 2024. doi: 10.18653/V1/2024.ACL-LONG.91. URL <https://doi.org/10.18653/v1/2024.acl-long.91>.

Jiazheng Kang, Mingming Ji, Zhe Zhao, and Ting Bai. Memory os of ai agent. *ArXiv*, abs/2506.06326, 2025. URL <https://api.semanticscholar.org/CorpusID:279250574>.

LangChain. Langmem sdk for agent long-term memory, 2025. URL <https://blog.langchain.com/langmem-sdk-launch/>.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In Hugo Larochelle, Marc’Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, and Hsuan-Tien Lin (eds.), *Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems 2020, NeurIPS 2020, December 6-12, 2020, virtual*, 2020. URL <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.

Hao Li, Chenghao Yang, An Zhang, Yang Deng, Xiang Wang, and Tat-Seng Chua. Hello again! llm-powered personalized agent for long-term dialogue, 2025a. URL <https://arxiv.org/abs/2406.05925>.

Yucheng Li, Bo Dong, Frank Guerin, and Chenghua Lin. Compressing context to enhance inference efficiency of large language models. In Houda Bouamor, Juan Pino, and Kalika Bali (eds.), *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, EMNLP 2023, Singapore, December 6-10, 2023*, pp. 6342–6353. Association for Computational Linguistics, 2023. doi: 10.18653/V1/2023.EMNLP-MAIN.391. URL <https://doi.org/10.18653/v1/2023.emnlp-main.391>.

Zhiyu Li, Shichao Song, Hanyu Wang, Simin Niu, Ding Chen, Jiawei Yang, Chenyang Xi, Huayi Lai, Jihao Zhao, Yezhaohui Wang, Junpeng Ren, Zehao Lin, Jiahao Huo, Tianyi Chen, Kai Chen, Ke-Rong Li, Zhiqiang Yin, Qingchen Yu, Bo Tang, Hongkang Yang, Zhiyang Xu, and Feiyu Xiong. Memos: An operating system for memory-augmented generation (mag) in large language models. *ArXiv*, abs/2505.22101, 2025b. URL <https://api.semanticscholar.org/CorpusID:278960153>.

Zongqian Li, Yin hong Liu, Yixuan Su, and Nigel Collier. Prompt compression for large language models: A survey. In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies, NAACL 2025 - Volume 1: Long Papers, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pp. 7182–7195. Association for Computational Linguistics, 2025c. doi: 10.18653/V1/2025.NAACL-LONG.368. URL <https://doi.org/10.18653/v1/2025.naacl-long.368>.

Xinnian Liang, Bing Wang, Huijia Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, and Zhou-jun Li. Scm: Enhancing large language model with self-controlled memory framework. 2023. URL <https://api.semanticscholar.org/CorpusID:258331553>.

Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, and Shane K. Luke. Prompt compression with context-aware sentence encoding for fast and improved LLM inference. In Toby Walsh, Julie Shah, and Zico Kolter (eds.), *AAAI-25, Sponsored by the Association for the Advancement of Artificial Intelligence, February 25 - March 4, 2025, Philadelphia, PA, USA*, pp. 24595–24604. AAAI Press, 2025. doi: 10.1609/AAAI.V39I23.34639. URL <https://doi.org/10.1609/aaai.v39i23.34639>.

蒋辉强, 吴乾辉, 罗旭方, 李东升, 林钦耀, 杨宇清, 邱丽丽. Longllmlingua: 通过提示压缩在长上下文场景中加速和增强LLM。收录于伦伟·库、安德烈·马丁斯和维韦克·斯里库马尔（编）的《计算语言学协会第62届年会论文集（第一卷：长篇论文）》，ACL 2024》，泰国曼谷，2024年8月11日至16日，第1658-1677页。计算语言学协会，2024年。doi: 10.18653/V1/2024.ACL-LONG.91。URL<https://doi.org/10.18653/v1/2024.acl-long.91>。

JiazhengKang, Mingming Ji, ZheZhao, 和 TingBai. AI代理的内存。 *ArXiv*,abs/2506.06326, 2025. URL<https://api.semanticscholar.org/CorpusID:279250574>.

LangChain. 代理长期记忆的LangMem SDK, 2025. URL<https://blog.langchain.com/langmem-sdk-launch/>.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, FabioPetroni, VladimirKarpukhin, NamanGoyal, Heinrich Küttler, MikeLewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, 和 Douwe Kiela. 用于知识密集型NLP任务的检索增强生成。在Hugo Larochelle, Marc’ Aurelio Ranzato, Raia Hadsell, Maria-Florina Balcan, 和 Hsuan-Tien Lin（编）的《神经信息处理系统进展33：2020年神经信息处理系统年度会议，NeurIPS 2020，2020年12月6-12日，线上》中，2020年。URL<https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.

郝立, 杨成浩, 张安, 邓阳, 王翔, 以及蔡天锡。再见了！基于大语言模型的个性化长期对话代理，2025a。URL<https://arxiv.org/abs/2406.05925>。

李玉成, 董波, 弗兰克·格伦, 以及林成华。压缩上下文以提升大语言模型的推理效率。在布阿莫·侯达, 皮诺·胡安, 和巴利·卡拉卡（编），《2023年自然语言处理经验方法会议论文集，EMNLP 2023》，新加坡，2023年12月6日至10日，第6342-6353页。计算语言学协会，2023年。doi: 10.18653/V1/2023.EMNLP-MAIN.391。URL<https://doi.org/10.18653/v1/2023.emnlp-main.391>。

李智宇, 宋诗超, 王汉宇, 牛思敏, 陈丁, 杨佳伟, 西晨阳, 赖华怡, 赵继浩, 王一舟, 任俊鹏, 林泽豪, 胡嘉豪, 陈天一, 陈凯, 李克荣, 殷志强, 余清尘, 唐波, 杨红康, 许志阳, 邢飞宇。Memos: 一种用于大语言模型内存增强生成（mag）的操作系统。 *ArXiv*, abs/2505.22101, 2025b. URL<https://api.semanticscholar.org/CorpusID:278960153>.

李宗乾, 刘银红, 苏奕璇, 和 Nigel Collier. 大语言模型的提示压缩：一项调查。收录于 Luis Chiruzzo, Alan Ritter, 和 王璐（编）, 2025年美洲国家计算语言学协会会议论文集：人机语言技术, NAACL 2025 - 卷1: 长篇论文, 新墨西哥州阿尔伯克基, 美国, 2025年4月29日 - 5月4日, 第7182-7195页。计算语言学协会，2025c. doi: 10.18653/V1/2025.NAACL-LONG.368. URL<https://doi.org/10.18653/v1/2025.naacl-long.368>.

Xinnian Liang, Bing Wang, Huijia Huang, Shuangzhi Wu, Peihao Wu, Lu Lu, Zejun Ma, 和 Zhou-jun Li. Scm: 使用自控记忆框架增强大型语言模型。 2023. URL <https://api.semanticscholar.org/CorpusID:258331553>.

Barys Liskavets, Maxim Ushakov, Shuvendu Roy, Mark Klibanov, Ali Etemad, 和 Shane K. Luke. 基于上下文感知句子编码的提示压缩技术，用于快速和改进的大语言模型推理。收录于 Toby Walsh, Julie Shah, 和 Zico Kolter（编）, AAAI-25, 由人工智能进步协会赞助，2025年2月25日至3月4日，美国宾夕法尼亚州费城，第24595–24604页。AAAI出版社，2025年。doi: 10.1609/AAAI.V39I23.34639. URL<https://doi.org/10.1609/aaai.v39i23.34639>。

Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, et al. Advances and challenges in foundation agents: From brain-inspired intelligence to evolutionary, collaborative, and safe systems. *arXiv preprint arXiv:2504.01990*, 2025a.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts, 2023. URL <https://arxiv.org/abs/2307.03172>.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Trans. Assoc. Comput. Linguistics*, 12:157–173, 2024. doi: 10.1162/TACL_A_00638. URL https://doi.org/10.1162/tacl_a_00638.

Zuhong Liu, Charles-Elie Simon, and Fabien Caspani. Passage segmentation of documents for extractive question answering. In Claudia Hauff, Craig Macdonald, Dietmar Jannach, Gabriella Kazai, Franco Maria Nardini, Fabio Pinelli, Fabrizio Silvestri, and Nicola Tonellotto (eds.), *Advances in Information Retrieval - 47th European Conference on Information Retrieval, ECIR 2025, Lucca, Italy, April 6-10, 2025, Proceedings, Part III*, volume 15574 of *Lecture Notes in Computer Science*, pp. 345–352. Springer, 2025b. doi: 10.1007/978-3-031-88714-7_33. URL https://doi.org/10.1007/978-3-031-88714-7_33.

Adyasha Maharana, Dong-Ho Lee, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri, and Yuwei Fang. Evaluating very long-term conversational memory of llm agents, 2024. URL <https://arxiv.org/abs/2402.17753>.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang, et al. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.

Jiayan Nan, Wenquan Ma, Wenlong Wu, and Yize Chen. Nemori: Self-organizing agent memory inspired by cognitive science. *arXiv preprint arXiv:2508.03341*, 2025.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders, and Joseph Gonzalez. Memgpt: Towards llms as operating systems. *ArXiv*, abs/2310.08560, 2023. URL <https://api.semanticscholar.org/CorpusID:263909014>.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Dongmei Zhang. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Findings of the Association for Computational Linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024*, pp. 963–981. Association for Computational Linguistics, 2024a. doi: 10.18653/V1/2024.FINDINGS-ACL.57. URL <https://doi.org/10.18653/v1/2024.findings-acl.57>.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Menglin Xia, Xufang Luo, Jue Zhang, Qingwei Lin, Victor Rühle, Yuqing Yang, Chin-Yew Lin, et al. Llmlingua-2: Data distillation for efficient and faithful task-agnostic prompt compression. *arXiv preprint arXiv:2403.12968*, 2024b.

Zhuoshi Pan, Qianhui Wu, Huiqiang Jiang, Xufang Luo, Hao Cheng, Dongsheng Li, Yuqing Yang, Chin-Yew Lin, H. Vicky Zhao, Lili Qiu, and Jianfeng Gao. Secom: On memory construction and retrieval for personalized conversational agents. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=xKDZAW0He3>.

Joon Sung Park, Joseph C. O’Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In Sean Follmer, Jeff Han, Jürgen Steimle, and Nathalie Henry Riche (eds.), *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology, UIST 2023, San Francisco, CA, USA, 29 October 2023- 1 November 2023*, pp. 2:1–2:22. ACM, 2023. doi: 10.1145/3586183.3606763. URL <https://doi.org/10.1145/3586183.3606763>.

Bang Liu, Xinfeng Li, Jiayi Zhang, Jinlin Wang, Tanjin He, Sirui Hong, Hongzhang Liu, Shaokun Zhang, Kaitao Song, Kunlun Zhu, 等. 基础智能体的进展与挑战：从类脑智能到进化、协作和安全系统. *arXiv预印本 arXiv:2504.01990*, 2025a.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 迷失在中间：语言模型如何使用长上下文, 2023. URL<https://arxiv.org/abs/2307.03172>.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 迷失在中间：语言模型如何使用长上下文. 计算语言学协会期刊, 12:157–173, 2024. doi: 10.1162/TACL_A_00638. URLhttps://doi.org/10.1162/tacl_a_00638.

Zuhong Liu, Charles-Elie Simon, 和 Fabien Caspani. 用于抽取式问答的文档片段分割。收录于 Claudia Hauff, Craig Macdonald, Dietmar Jannach, Gabriella Kazai, Franco Maria Nardini, Fabio Pinelli, Fabrizio Silvestri, 和 Nicola Tonellotto (编), 信息检索进展 - 第 47 届欧洲信息检索会议, *ECIR 2025*, 意大利卢卡, 2025 年 4 月 6-10 日, 会议录, 第三部分, 第 15574 卷的 计算机科学讲座笔记, pp. 345–352.Springer, 2025b. doi: 10.1007/978-3-031-88714-7_33. URLhttps://doi.org/10.1007/978-3-031-88714-7_33.

Adyasha Maharana, 东浩李, Sergey Tulyakov, Mohit Bansal, Francesco Barbieri 和 Yuwei Fang. 评估大语言模型代理的极长期对话记忆, 2024. URL<https://arxiv.org/abs/2402.17753>.

Lingrui Mei, Jiayu Yao, Yuyao Ge, Yiwei Wang, Baolong Bi, Yujun Cai, Jiazhi Liu, Mingyu Li, Zhong-Zhi Li, Duzhen Zhang 等. 大语言模型上下文工程的综述.*arXiv 预印本 arXiv:2507.13334*, 2025.

Jiayan Nan, Wenquan Ma, Wenlong Wu 和 Yize Chen. Nemori: 受认知科学启发的自组织代理记忆. *arXiv 预印本 arXiv:2508.03341*, 2025.

Charles Packer, Vivian Fang, Shishir G. Patil, Kevin Lin, Sarah Wooders 和 Joseph Gonzalez. Memgpt: 使大语言模型成为操作系统.*ArXiv*, abs/2310.08560, 2023. URL <https://api.semanticscholar.org/CorpusID:263909014>.

朱仕潘, 吴乾辉, 蒋会强, 夏梦琳, 罗旭方, 张决, 林清伟, Victor R`乌勒, 杨雨晴, 林钦耀, 赵慧薇, 邱丽丽, 张冬梅. LLMLingua-2: 数据蒸馏用于高效且忠实于任务的提示压缩。在陆伟, 安德烈·马丁斯, 和Vivek Srikumar (编), 计算语言学协会协会发现, *ACL 2024*, 泰国曼谷和线上会议, 2024年8月11-16日, 第963-981页。计算语言学协会, 2024a. doi: 10.18653/V1/2024.FINDINGS-ACL.57. URL<https://doi.org/10.18653/v1/2024.findings-acl.57>。

朱仕潘, 吴乾辉, 蒋会强, 夏梦琳, 罗旭方, 张决, 林清伟, Victor R`乌勒, 杨雨晴, 林钦耀, 等。LLMLingua-2: 数据蒸馏用于高效且忠实于任务的提示压缩. *arXiv预印本 arXiv:2403.12968*, 2024b。

朱仕潘, 吴乾辉, 蒋慧强, 罗旭方, 程浩, 李东升, 杨宇清, 林振宇, 赵慧薇, 邱丽丽, 高建锋. Secom: 关于个性化对话代理的记忆构建与检索。在第13届国际学习表示会议, *ICLR 2025*, 新加坡, 2025年4月24-28日。OpenReview.net, 2025年。URL <https://openreview.net/forum?id=xKDZAW0He3>。

Joon Sung Park, Joseph C. O’ Brien, Carrie Jun Cai, Meredith Ringel Morris, Percy Liang 和 Michael S. Bernstein. 生成式代理：人类行为的交互式拟像。在 Sean Follmer, Jeff Han, Jürgen Steimle 和 Nathalie Henry Riche (eds.) 主编的《第 36 届 ACM 用户界面软件与技术研讨会论文集, UIST 2023, 美国加利福尼亚州旧金山, 2023 年 10 月 29 日 - 11 月 1 日》, 第 2:1–2:22 页。ACM, 2023. doi: 10.1145/3586183.3606763. URL`https://doi.org/10.1145/3586183.3606763。

Renyi Qu, Ruixuan Tu, and Forrest Sheng Bao. Is semantic chunking worth the computational cost? In Luis Chiruzzo, Alan Ritter, and Lu Wang (eds.), *Findings of the Association for Computational Linguistics: NAACL 2025, Albuquerque, New Mexico, USA, April 29 - May 4, 2025*, pp. 2155–2177. Association for Computational Linguistics, 2025. doi: 10.18653/V1/2025.FINDINGS-NAACL.114. URL <https://doi.org/10.18653/v1/2025.findings-naacl.114>.

Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: a temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.

Alireza Rezazadeh, Zichao Li, Wei Wei, and Yujia Bao. From isolated conversations to hierarchical schemas: Dynamic tree memory representation for llms. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=moXtEmCleY>.

Rana Salama, Jason Cai, Michelle Yuan, Anna Currey, Monica Sunkara, Yi Zhang, and Yassine Benajiba. Meminsight: Autonomous memory augmentation for llm agents. *ArXiv*, abs/2503.21760, 2025. URL <https://api.semanticscholar.org/CorpusID:277349587>.

Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. RAPTOR: recursive abstractive processing for tree-organized retrieval. In *The Twelfth International Conference on Learning Representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*. OpenReview.net, 2024. URL <https://openreview.net/forum?id=GN921JHCRw>.

Zhen Tan, Jun Yan, I-Hung Hsu, Rujun Han, Zifeng Wang, Long T. Le, Yiwen Song, Yanfei Chen, Hamid Palangi, George Lee, Anand Iyer, Tianlong Chen, Huan Liu, Chen-Yu Lee, and Tomas Pfister. In prospect and retrospect: Reflective memory management for long-term personalized dialogue agents, 2025. URL <https://arxiv.org/abs/2503.08026>.

Yu Wang and Xi Chen. Mirix: Multi-agent memory system for llm-based agents. *arXiv preprint arXiv:2507.07957*, 2025.

Yu Wang, Chi Han, Tongtong Wu, Xiaoxin He, Wangchunshu Zhou, Nafis Sadeq, Xiusi Chen, Zexue He, Wei Wang, Gholamreza Haffari, et al. Towards lifespan cognitive systems. *arXiv preprint arXiv:2409.13265*, 2024.

Jason Weston and Sainbayar Sukhbaatar. System 2 attention (is something you might need too). *arXiv preprint arXiv:2311.11829*, 2023.

Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Longmemeval: Benchmarking chat assistants on long-term interactive memory. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025. URL <https://openreview.net/forum?id=pZiyCaVuti>.

Heming Xia, Chak Tou Leong, Wenjie Wang, Yongqi Li, and Wenjie Li. Tokenskip: Controllable chain-of-thought compression in llms. *arXiv preprint arXiv:2502.12067*, 2025.

Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-mem: Agentic memory for llm agents. *ArXiv*, abs/2502.12110, 2025. URL <https://api.semanticscholar.org/CorpusID:276421617>.

Hongkang Yang, Zehao Lin, Wenjin Wang, Hao Wu, Zhiyu Li, Bo Tang, Wenqiang Wei, Jinbo Wang, Zeyun Tang, Shichao Song, Chenyang Xi, Yu Yu, Kai Chen, Feiyu Xiong, Linpeng Tang, and Weinan E. Memory³: Language modeling with explicit memory. *CoRR*, abs/2407.01178, 2024. doi: 10.48550/ARXIV.2407.01178. URL <https://doi.org/10.48550/arXiv.2407.01178>.

Guibin Zhang, Muxin Fu, Guancheng Wan, Miao Yu, Kun Wang, and Shuicheng Yan. G-memory: Tracing hierarchical memory for multi-agent systems. *arXiv preprint arXiv:2506.07398*, 2025.

Zeyu Zhang, Xiaohe Bo, Chen Ma, Rui Li, Xu Chen, Quanyu Dai, Jieming Zhu, Zhenhua Dong, and Ji-Rong Wen. A survey on the memory mechanism of large language model based agents, 2024. URL <https://arxiv.org/abs/2404.13501>.

Renyi Qu, Ruixuan Tu 和 Forrest Sheng Bao. 语义分块值得计算成本吗? 在 Luis Chiruzzo, Alan Ritter 和 Lu Wang (eds.) 主编的《计算语言学协会发现: NAACL 2025, 美国新墨西哥州阿尔伯克基, 2025 年 4 月 29 日 - 5 月 4 日》, 第 2155–2177 页. 计算语言学协会, 2025. doi: 10.18653/V1/2025.FINDINGS-NAACL.114. URL <https://doi.org/10.18653/v1/2025.findings-naacl.114>.

普雷斯頓·拉斯穆森、帕夫洛·帕利伊丘克、特拉維斯·博瓦、杰克·瑞安和丹尼尔·查莱夫。Zep: 一种用于代理内存的时间知识图谱架构。arXiv预印本 arXiv:2501.13956, 2025年。阿里雷扎·雷扎扎德、李志超、魏伟和包宇嘉。从隔离对话到分层模式: 用于大语言模型的动态树形内存表示。在第十三届学习表示国际会议, ICLR 2025, 新加坡, 2025年4月24日至28日。OpenReview.net, 2025年。URL <https://openreview.net/forum?id=moXtEmCleY>。拉娜·萨拉马、贾森·蔡、袁美琪、安娜·卡里、莫妮卡·桑卡拉、张怡和贝纳吉巴·亚辛。Meminsight: 用于大语言模型代理的自主内存增强。ArXiv, abs/2503.21760, 2025年。URL <https://api.semanticscholar.org/CorpusID:277349587>。帕尔萨·萨西、萨拉曼·阿卜杜拉、阿迪蒂·图利、舒布·卡纳、安娜·戈尔迪和克里斯托弗·D·曼宁。RAPTOR: 用于树形组织检索的递归抽象处理。在第十二届学习表示国际会议, ICLR 2024, 奥地利维也纳, 2024年5月7日至11日。OpenReview.net, 2024年。URL <https://openreview.net/forum?id=GN921JHCRw>。谭振、闫俊、许一宏、韩如君、王子峰、黎隆泰、宋艺文、陈岩飞、帕尔马吉·哈密德、乔治·李、伊亚南、陈天龙、刘欢、李晨宇和托马斯·普菲斯特。展望与回顾: 长期个性化对话代理的反思性内存管理, 2025年。URL <https://arxiv.org/abs/2503.08026>。王宇和陈希。Mirix: 基于大语言模型的代理的多智能体内存系统。arXiv预印本 arXiv:2507.07957, 2025年。王宇、韩驰、吴通通、何晓欣、周王春舒、萨迪克·纳菲斯、陈秀思、何泽雪、王伟、哈菲阿里·格哈姆雷扎等。迈向生命周期认知系统。arXiv预印本 arXiv:2409.13265, 2024年。贾森·韦斯顿和赛恩巴亚尔·苏克巴塔。系统2注意力(你可能也需要这个)。arXiv预印本 arXiv:2311.11829, 2023年。吴迪、王宏伟、余文豪、张宇伟、张凯伟和余东。Longmemeval: 在长期交互记忆上测试聊天助手。在第十三届学习表示国际会议, ICLR 2025, 新加坡, 2025年4月24日至28日。OpenReview.net, 2025年。URL <https://openreview.net/forum?id=pZiyCaVuti>。夏铭、梁卓杰、李凯梅、高航、谭俊涛和张永峰。Tokenskip: 大语言模型中可控的思维链压缩。arXiv预印本 arXiv:2502.12067, 2025年。徐无江、梁祖杰、梅凯、高航、谭俊涛和张永峰。A-mem: 用于大语言模型代理的代理内存。ArXiv, abs/2502.12110, 2025年。URL <https://api.semanticscholar.org/CorpusID:276421617>。杨宏康、林哲浩、王文进、吴浩、李志宇、唐波、魏文强、王金波、唐泽云、宋时超、西晨阳、王宇、陈凯、熊飞宇、唐林鹏和刘伟南。Memory³: 具有显式内存的语言建模。CoRR, abs/2407.01178, 2024年。doi: 10.48550/ARXIV.2407.01178. URL <https://doi.org/10.48550/arXiv.2407.01178>。张桂斌、付木鑫、万观成、余苗、王坤和闫水成。G-memory: 多智能体系统的分层内存跟踪。arXiv预印本 arXiv:2506.07398, 2025年。张泽宇、薄晓鹤、马晨、李瑞、陈旭、戴全宇、朱继明、董振华和温继荣。关于基于大语言模型的代理的内存机制的调查, 2024年。URL <https://arxiv.org/abs/2404.13501>。

Jihao Zhao, Zhiyuan Ji, Yuchen Feng, Pengnian Qi, Simin Niu, Bo Tang, Feiyu Xiong, and Zhiyu Li. Meta-chunking: Learning text segmentation and semantic completion via logical perception. 2024. URL <https://api.semanticscholar.org/CorpusID:278782541>.

Yibo Zhao, Jiapeng Zhu, Ye Guo, Kangkang He, and Xiang Li. E²graphrag: Streamlining graph-based rag for high efficiency and effectiveness. *arXiv preprint arXiv:2505.24226*, 2025.

Wanjun Zhong, Lianghong Guo, Qiqi Gao, He Ye, and Yanlin Wang. Memorybank: Enhancing large language models with long-term memory. In Michael J. Wooldridge, Jennifer G. Dy, and Sriraam Natarajan (eds.), *Thirty-Eighth AAAI Conference on Artificial Intelligence, AAAI 2024, Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence, IAAI 2024, Fourteenth Symposium on Educational Advances in Artificial Intelligence, EAAI 2024, February 20-27, 2024, Vancouver, Canada*, pp. 19724–19731. AAAI Press, 2024. doi: 10.1609/AAAI.V38I17.29946. URL <https://doi.org/10.1609/aaai.v38i17.29946>.

赵继浩、姬志远、冯宇晨、戚鹏年、牛思敏、唐波、熊飞宇和李志宇。元块：通过逻辑感知学习文本分割和语义补全。2024年。URL<https://api.semanticscholar.org/CorpusID:278782541>。

赵毅博、朱嘉鹏、郭晔、何康康和李祥。E²graphrag：简化基于图的rag以实现高效率和有效性。*arXiv* 预印本 *arXiv:2505.24226*, 2025。

钟万军，郭良红，高琪琪，叶鹤，和王延林。Memorybank：通过长期记忆增强大型语言模型。在迈克尔·J·伍德布里奇、詹妮弗·G·戴和斯里拉姆·纳塔拉jan（编），第三十八届AAAI人工智能会议，*AAAI 2024*，第三十六届人工智能创新应用会议，*IAAI 2024*，第十四届人工智能教育进步研讨会，*EAAI 2024*，2024年2月20-27日，加拿大温哥华，第19724-19731页。AAAI出版社，2024。doi: 10.1609/AAAI.V38I17.29946。URL<https://doi.org/10.1609/aaai.v38i17.29946>。

A BACKGROUND DETAILS

A.1 BACKGROUND ABOUT CURRENT MEMORY SYSTEMS

We describe both the mainstream memory architectures and the **LightMem** pipeline in terms of two major stages. The first is the memory bank construction stage, which can be further decomposed into the three sub-stages (I), (II), and (III) described in the Section 2.1. The second major stage concerns the usage of the memory system, which consists of retrieval and question answering (QA).

Memory Bank Construction As shown in Table 4, we detail the workflows of the three sub-stages (I), (II), and (III) for naive RAG, prevailing memory systems, and our LightMem. It can be observed that baseline memory systems typically perform their update stage during user-model interaction, which introduces substantial test-time latency. In contrast, LightMem decouples this update process from online interaction, thereby significantly reducing test-time latency. All models involved in these processes are listed in Table 5. As shown, LightMem introduces only one additional model, LLMlingua-2, beyond those used by baseline methods. This model follows a lightweight BERT architecture and requires less than 2GB of GPU memory during inference, rendering its overhead negligible. Moreover, for fairness, the latency introduced by this component is fully accounted for in our reported Runtime metric.

Table 4: The mainstream memory architectures and the LightMem pipeline of memory bank construction stage. Black-font processes denote those executed during online test-time interactions, whereas **red-font** processes denote those executed offline.

Method	(I) Segment	(II) Summary/Extrct	(III) Update
NaiveRAG	Raw dialog $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{seg}_i\}$	$\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{emb}_i\}$	\
Other Memory Systems	Raw dialog $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{seg}_i\}$	$\rightarrow f_{\text{sum/extract}}()$ $\rightarrow \{\text{memory entry}_i\}$ $\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{emb}_i\}$	$\rightarrow f_{\text{retrieve}}()$ $\rightarrow \{\text{related entry}_i\}$ $\rightarrow f_{\text{update}}()$ $\rightarrow \{\text{add, delete, update, merge...}\}$
LightMem	Raw dialog $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{seg}_i\}$ $\rightarrow f_{\text{pre_compress}}()$ $\rightarrow \{\text{comp_seg}_i\}$ $\rightarrow \text{sensory buffer full} \rightarrow$ $f_{\text{topic}}() \rightarrow$ $\{\text{topic-wise comp_seg}_i\}$	$\rightarrow \text{STM buffer full} \rightarrow f_{\text{sum/extract}}()$ $\rightarrow \{\text{topic}_i, \{\text{memory entry}_j\}\}$ $\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{topic}_i, \{\text{emb}_j\}\}$	Offline update trigger $\{\text{every entry}_i\} \rightarrow f_{\text{retrieve}}()$ $\rightarrow \{\text{related entry}_j\} \rightarrow \{\text{update queue}\}$ All update queues established $\rightarrow \text{parallel } f_{\text{update}}()$ $\rightarrow \{\text{add, delete, update, merge...}\}$

Function	Model / Strategy	Implementation in This Paper
$f_{\text{seg}}()$	Segmentation strategy	Turn-level granularity input
$f_{\text{index}}()$	Embedding model	all-MiniLM-L6-v2
$f_{\text{sum/extract}}()$	System backbone model	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507
$f_{\text{retrieve}}()$	Retrieval strategy	Cosine similarity vector retrieval
$f_{\text{update}}()$	System backbone model	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507
$f_{\text{pre_compress}}()$	Token compression model	LLMlingua-2
$f_{\text{topic}}()$	Topic segmentation model	LLMlingua-2
$f_{\text{chat}}()$	Chat model	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507

Table 5: Mapping between functions, their roles, and the concrete models used in this paper. Black-font entries denote models shared by both LightMem and baseline methods, whereas **red-font** entries denote models unique to LightMem.

Retrieval and Usage After the memory bank construction stage, we obtain an up-to-date memory bank. When a new user query arrives, the memory system use $f_{\text{retrieve}}()$ to retrieve relevant entries from this repository, appends them to the query, and then prompts the chat model $f_{\text{chat}}()$ to produce a response.

背景详情

A.1 背景关于当前内存系统

我们描述了主流内存架构和**LightMem**流程，分为两个主要阶段。第一阶段是内存库构建阶段，可以进一步分解为第2.1节中描述的三个子阶段（I）、（II）和（III）。第二阶段主要涉及内存系统的使用，包括检索和问答（QA）。

内存库构建 如表所示4，我们详细介绍了朴素RAG、主流内存系统和我们的LightMem三个子阶段（I）、（II）和（III）的工作流程。可以看出，基线内存系统通常在用户-模型交互期间执行更新阶段，这会引入显著的测试时延迟。相比之下，LightMem将此更新过程与在线交互解耦，从而显著降低了测试时延迟。所有参与这些过程的模型都列在表 5中。如图所示，LightMem在基线方法使用的模型之外仅引入了一个额外的模型，即LLMLingua-2。该模型遵循轻量级BERT架构，在推理期间需要不到2GB的GPU内存，因此其开销可以忽略不计。此外，为了公平性，该组件引入的延迟在我们的报告Runtime指标中得到了完全考虑。

表4：主流内存架构和LightMem内存库构建阶段的流程。黑体字表示在在线测试时交互期间执行的过程，而**红体字**表示离线执行的过程。

方法	(I) 段	(II) 摘要/提取	(III) 更新
NaiveRAG	原始对话 $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{seg}_i\}$	$\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{emb}_i\}$	\
其他内存系统	原始对话 $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{seg}_i\}$	$\rightarrow f_{\text{sum/extract}}()$ $\rightarrow \{\text{内存条目}_i\}$ $\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{emb}_i\}$	$\rightarrow f_{\text{retrieve}}()$ $\rightarrow \{\text{相关条目}_i\}$ $\rightarrow f_{\text{update}}()$ $\rightarrow \{\text{添加, 删除, 更新, 合并...}\}$
LightMem	原始对话 $\rightarrow f_{\text{seg}}()$ $\rightarrow \{\text{段}_i\}$ $\rightarrow f_{\text{pre_compress}}()$ $\rightarrow \{\text{合并段}_i\}$ $\rightarrow \text{感觉缓冲区已满} \rightarrow$ $f_{\text{topic}}() \rightarrow$ $\{\text{按主题的复合分段}_i\}$	$\rightarrow \text{STM缓冲区已满} \rightarrow f_{\text{sum/extract}}()$ $\rightarrow \{\text{主题}_i, \{\text{内存条目}_j\}\}$ $\rightarrow f_{\text{index}}()$ $\rightarrow \{\text{主题}_i, \{\text{嵌入}_j\}\}$	离线更新触发 $\{\text{每个条目}_i\} \rightarrow f_{\text{retrieve}}()$ $\rightarrow \{\text{相关条目}_j\} \rightarrow \{\text{更新队列}\}$ 所有更新队列已建立 $\rightarrow \text{并行 } f_{\text{update}}()$ $\rightarrow \{\text{添加、删除、更新、合并...}\}$

函数	模型 / 策略	本文中的实现
$f_{\text{seg}}()$	分割策略	回合级粒度输入
$f_{\text{index}}()$	嵌入模型	all-MiniLM-L6-v2
$f_{\text{sum/extract}}()$	系统骨干模型	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507
$f_{\text{retrieve}}()$	检索策略	余弦相似度向量检索
$f_{\text{update}}()$	系统骨干模型	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507
$f_{\text{pre_压缩}}()$	token压缩模型	LLMLingua-2
$f_{\text{topic}}()$	主题分割模型	LLMLingua-2
$f_{\text{chat}}()$	聊天模型	GPT-4o-mini; Qwen3-30B-A3B-Instruct-2507

表5：函数、其角色以及本文中使用的具体模型之间的映射。黑色字体条目表示LightMem和基线方法共享的模型，而**红色字体**条目表示LightMem独有的模型。

检索和使用在内存库构建阶段之后，我们获得一个最新的内存库。当一个新的用户查询到达时，内存系统使用 $f_{\text{retrieve}}()$ 从该库中检索相关条目，将它们附加到查询中，然后提示聊天模型 $f_{\text{chat}}()$ 生成响应。

A.2 NOTATION AND COMPLEXITY DETAILS

Table 6: Notation used in complexity analysis (§Section 4).	
Symbol	Definition
N	Total number of turns in a dialogue history.
T	Average number of tokens per turn.
r	Token compression rate (as defined in the main paper). After one compression step, only a fraction r of tokens is retained.
x	Number of compression iterations. In LightMem, the <i>pre-compress</i> module may be invoked multiple times for the same message to remove redundancy until the message is sufficiently compact. This occurs frequently in datasets such as LongMemEval . All time costs are included in runtime metrics.
th	Capacity of the Short-Term Memory (STM) buffer, as defined in the paper.
$L_{\text{sum-in}} / L_{\text{sum-out}}$	Number of tokens in the input prompt template and output of a single backbone LLM call for <i>summarization</i> . These are similar across memory frameworks.
M_1 / M_2	Number of memory entries produced from a single summarization operation under Other Memory Systems (M_1) and LightMem (M_2).
$L_{\text{up-in}} / L_{\text{up-out}}$	Number of tokens in the input prompt template and output of a single backbone LLM call for <i>memory update</i> . Similar across frameworks.
R_1 / R_2	Proportion of summary entries that successfully retrieve at least one relevant memory entry (triggering an update) for Other Memory Systems (R_1) and LightMem (R_2). Some entries do not retrieve any relevant counterparts and thus do not trigger updates.

B USAGE OF LLMs

Throughout the preparation of this manuscript, we used LLMs to assist with improving grammar, clarity, and wording in parts of this work. The use of LLMs was limited to language refinement, with all ideas, analyses, and conclusions solely developed by the authors.

C METHODOLOGY DETAILS

C.1 TOPIC SEGMENTATION

In this part, we present the construction of the attention matrix, the underlying rationale for topic segmentation, and representative illustrative cases.

We extract only the user sentences from multi-turn dialogues, as they are generally more concise and the assistant’s responses necessarily remain consistent with the user’s theme within the same turn. Moreover, since the maximum input length of the LLMLingua-2 Pan et al. (2024b) model is 512 tokens, the assistant’s often lengthy sentences cannot be effectively accommodated. Therefore, we sequentially store the user sentences into a buffer and segment them, ensuring that as many sentences as possible are preserved while staying within the token limit. As a practical trick, if a sentence becomes empty after compression, we retain its original uncompressed version; if the token length of a sentence still exceeds the maximum limit, we continue to compress it using the LLMLingua-2 model at a 0.5 compression rate until the token length falls below the threshold. To reduce the effect of attention sinks, we mask out the contributions of the first and last three tokens in each sequence and subsequently normalize the remaining attention values. Attention is derived from the higher layers of LLMLingua-2 (layers 8, 9, 10, and 11). For any two sentences, we first compute token-level pairwise attention and average across tokens to obtain the overall attention of one sentence to the target sentence; we then average across the selected layers to obtain a more robust inter-sentence attention score. For each current sentence, the attention scores directed toward all

A.2 符号与复杂度细节

表6：复杂度分析中使用的符号（§ 第4节4）。	
符号	定义
N	对话历史中的总轮数
T	每轮的平均token数。
r	token压缩率（如主论文中定义）。压缩一步后，仅保留 r 部分token。
x	压缩迭代次数。在LightMem中，预压缩 模块可能多次为同一消息被调用以消除冗余，直到消息足够紧凑。这在 LongMemEval 等数据集中频繁发生。所有时间成本都包含在运行时指标中。
th	短期记忆（STM）缓冲区的容量，如论文中所述。
$L_{\text{sum-in}} / L_{\text{sum-out}}$	单个主干中 输入提示模板 和 输出 中的令牌数量 大语言模型（LLM）调用摘要。这些在内存框架中是相似的。
M_1 / M_2	单个摘要操作在内存系统下生成的条目数量 其他内存系统（ M_1 ）和 LightMem（ M_2 ）。
$L_{\text{up-in}} / L_{\text{up-out}}$	单个主干模型中 输入提示模板 和 输出 中的token数量 大语言模型（LLM）调用内存更新时的LLM调用。跨框架相似。
R_1 / R_2	成功检索到至少一条相关内存的摘要条目比例 条目（触发更新）用于其他内存系统（ R_1 ）和 LightMem（ R_2 ）。 部分条目未检索到任何相关对应项，因此不会触发更新。

B 大语言模型（LLM）的使用

在整个手稿的准备过程中，我们使用大语言模型来协助改进本文部分内容的语法、清晰度和措辞。大语言模型的使用仅限于语言润色，所有想法、分析和结论均由作者独立开发。

C 方法论细节

C.1 主题分割

在这一部分，我们介绍了注意力矩阵的构建、主题分割的底层原理以及具有代表性的说明性案例。

我们仅从多回合对话中提取用户句子，因为它们通常更简洁，且助手在同一回合中的回复必然与用户的主题保持一致。此外，由于LLMLingua-2Pan等人 (2024b) 模型的最大输入长度为512个token，助手的通常冗长的句子无法得到有效容纳。因此，我们将用户句子依次存储到缓冲区中并进行分段，确保尽可能多地保留句子，同时保持在token限制范围内。作为一个实用技巧，如果压缩后句子变为空，我们保留其原始未压缩版本；如果句子的token长度仍然超过最大限制，我们继续使用LLMLingua-2模型以0.5的压缩率进行压缩，直到token长度低于阈值。为了减少注意力陷阱的影响，我们遮蔽掉每个序列中第一个和最后三个token的贡献，并随后对剩余的注意力值进行归一化。注意力来自LLMLingua-2的高层（第8层、第9层、第10层和第11层）。对于任意两个句子，我们首先计算token级别的成对注意力，并对token进行平均以获得一个句子对目标句子的整体注意力；然后对选定的层进行平均，以获得更稳健的句子间注意力分数。对于每个当前句子，指向所有

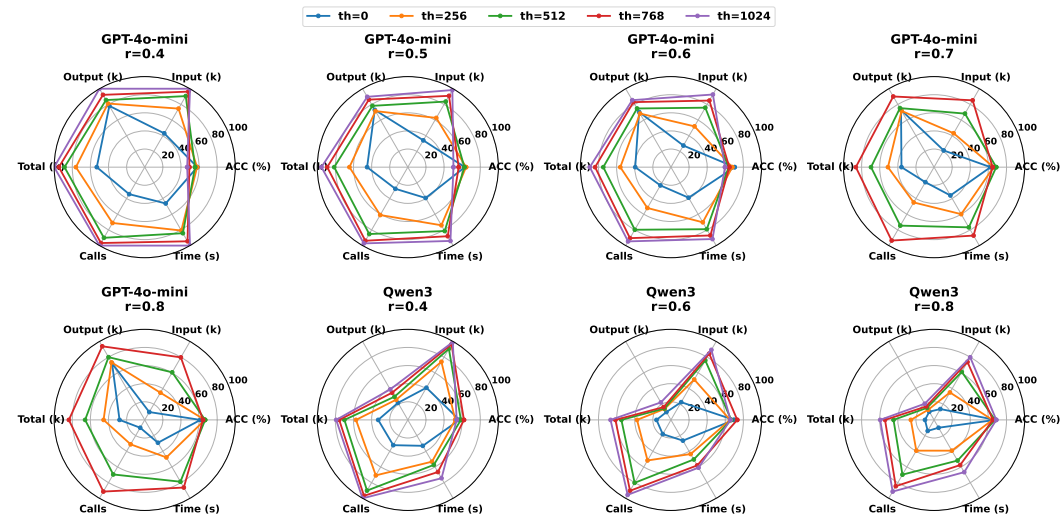


Figure 4: Impact of the STM buffer threshold (th) on performance and efficiency across different compression ratios (r). Each radar chart represents a specific configuration of a model (GPT-4o-mini or Qwen3) and a fixed compression ratio. The axes measure six key metrics: Accuracy (ACC), token consumption (Input, Output, Total), API Calls, and Runtime. To facilitate comparison, all values are normalized for visualization on the chart.

preceding sentences are normalized within the sentence, yielding the final attention matrix. Residual fragments that remain after segmentation are carried over to the beginning of the next buffer for further processing, and this procedure continues iteratively until the dialogue ends.

Based on the attention pattern, we focus on the sequence formed by each sentence’s attention scores relative to its immediately preceding sentence, which directly reflects the continuity of local semantics. Therefore, we take the attention scores from the outermost layer of the attention map. When the attention score at a given position is higher than both its preceding and following positions, it is regarded as a local peak. If a sentence is identified as a peak, we set a segmentation point immediately before this sentence, making the peak sentence the beginning of a new segment. The rationale is that the peak sentence exhibits consistently low attention to all earlier sentences overall and reflects a clear transition from an old topic to a new one, indicating that the identified sentence marks the initiation of a new topic.

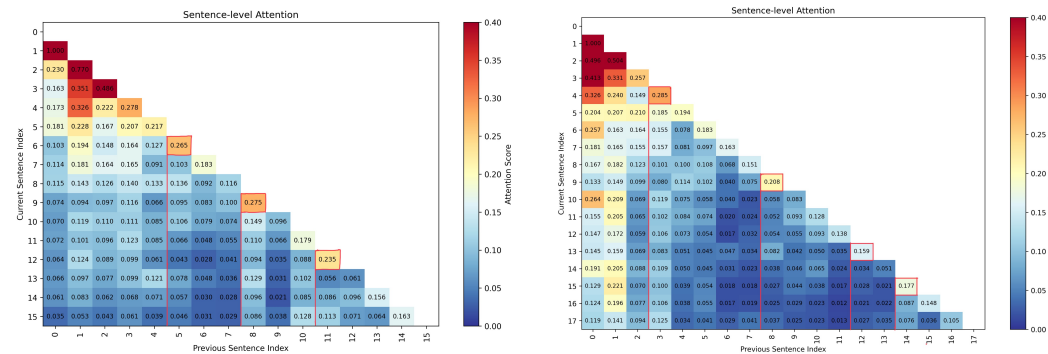


Figure 5: Example of Topic Segment Attention Matrix.

Figure 5 illustrates three representative examples of reliable segmentation under 50% compression rate. In the first attention map, local peaks in the adjacent-sentence attention sequence appear at positions 5, 8, and 11, where the actual segmentation boundaries lie between sentences 4–5 and 11–12.

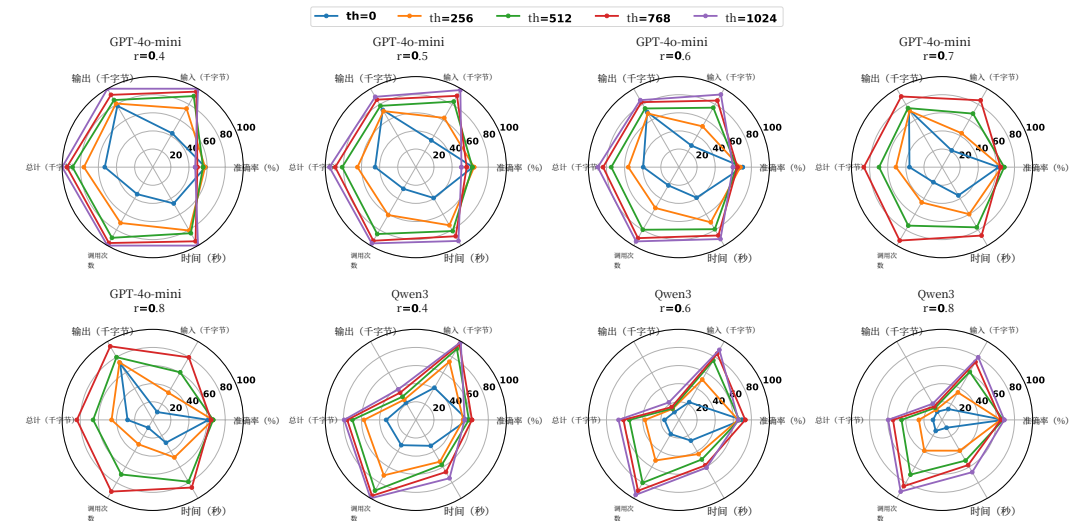


图4: STM缓冲区阈值(th)对性能和效率在不同压缩率(r)下的影响。每个雷达图代表一个特定模型（GPT-4o-mini或Qwen3）和固定压缩率的配置。坐标轴衡量六个关键指标：准确率（ACC）、token消耗（输入、输出、总计）、API调用次数和运行时间。为了便于比较，图表中的所有值都已归一化进行可视化。

残差片段会被带到下一个缓冲区的开头以进行进一步处理，该过程持续迭代，直到对话结束。

根据注意力模式，我们关注由每句话的注意力分数相对于其紧邻的前一句形成的序列，这直接反映了局部语义的连续性。因此，我们从注意力图的outermost层获取注意力分数。当某个位置的注意力分数同时高于其前一个和后一个位置时，它被视为一个局部峰值。如果一句话被识别为峰值，我们在该句之前立即设置一个分割点，使峰值句成为新片段的开始。其原理是，峰值句在整体上对所有更早的句子表现出持续的低注意力，并反映出一个旧主题到新主题的清晰过渡，表明识别出的句子标志着新主题的起始。

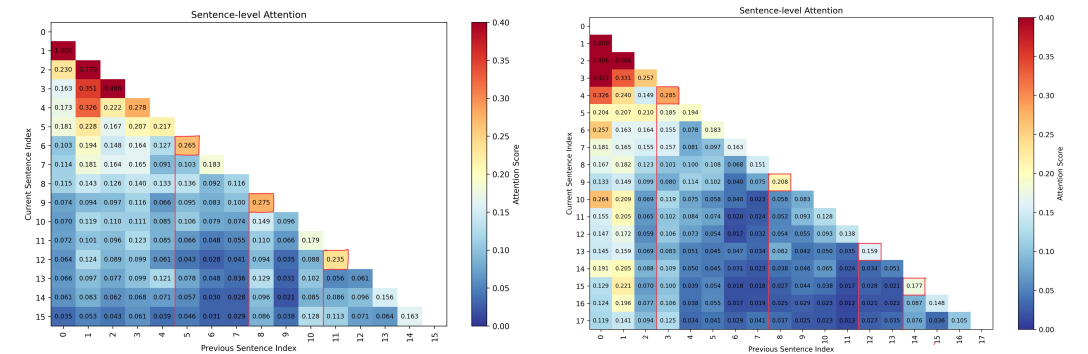


图5：主题分割注意力矩阵示例。

图5展示了在50%压缩率下三种可靠的分割代表性示例。在第一个注意力图中，相邻句子注意力序列中的局部峰值出现在位置5、8和11处，而实际分割边界位于句子4–5和11–12之间。

In the second attention map, peaks occur at positions 3, 8, 12, and 14, and the actual boundaries are located between sentences 7–8, 11–12, and 13–14. Overall, our method achieves close alignment with the majority of true boundaries while providing finer-grained segmentation. These examples demonstrate that our segmentation approach enables both fine-grained and reliable detection of topic boundaries, thereby validating its effectiveness.

C.2 CATEGORY-WISE ACCURACY

As summarized in Table 7, retrieval-augmented and memory-centric methods (e.g., *A-MEM*, *Mem0*, *MemoryOS*) generally outperform *Full Text* on categories that demand information integration or belief revision, such as *Temporal*, *Multi-Session*, and *Knowledge-Update*. In contrast, categories such as *Single-User* and *Single-Assistant*, lightweight retrieval like *Naive RAG* is often competitive and can be the most reliable option, while *Single-Preference* shows higher variance due to its smaller sample size.

Table 7: **Category-wise Accuracy.** Accuracy (%) by method across question types. Parentheses indicate category proportion and sample size. For GPT, LightMem is configured with parameters $r = 0.7$ and $th = 512$; for Qwen, LightMem is configured with $r = 0.4$ and $th = 768$.

Method	Temporal ($n=133$)	Multi-Session ($n=133$)	Knowledge-Update ($n=78$)	Single-User ($n=70$)	Single-Assistant ($n=56$)	Single-Preference ($n=30$)
 GPT-4o-mini						
<i>Full Text</i>	31.58	45.45	76.92	87.14	89.29	36.67
<i>Naive RAG</i>	39.85	48.48	67.95	90.00	98.21	53.33
<i>LangMem</i>	15.79	20.30	66.67	60.00	46.43	60.00
<i>A-MEM</i>	47.36	48.87	64.11	92.86	96.43	46.67
<i>MemoryOS</i>	32.33	31.06	48.72	80.00	64.29	30.00
<i>Mem0</i>	40.15	46.21	70.12	81.43	41.07	60.00
<i>LightMem</i>	67.18	71.74	83.12	87.14	32.14	68.18
 Qwen3-30B-A3B-Instruct-2507						
<i>Full Text</i>	33.08	35.61	76.92	82.86	87.50	50.00
<i>Naive RAG</i>	36.84	47.73	65.38	91.43	98.21	70.00
<i>LangMem</i>	37.60	38.35	67.95	78.57	42.86	70.00
<i>A-MEM</i>	51.88	51.12	76.93	90.00	96.43	40.00
<i>MemoryOS</i>	28.57	36.84	61.54	72.86	92.86	33.33
<i>Mem0</i>	41.94	28.13	28.57	55.32	26.09	81.82
<i>LightMem</i>	54.20	51.91	66.67	80.00	31.25	80.00

C.3 DETAILED PARAMETER ANALYSIS

As Table 9 shows, we report the numerical results of the effects of LightMem parameters (compression ratio r and STM threshold th).

D EXPERIMENT DETAILS

D.1 DATASETS AND BASELINES

Datasets The LongMemEval dataset (Wu et al., 2025) is designed to benchmark long-term interactive memory in conversational agents. It comprises 500 evaluation questions built upon extended user-assistant dialogues. It has two different versions with different lengths: the LONGMEMEVAL-S setting contains approximately 115k tokens per problem, while the LONGMEMEVAL-M setting extends up to 1.5 million tokens across 500 sessions. In our work, we adopt the LONGMEMEVAL-S version due to its balance between dialogue length and computational feasibility. The questions are categorized into multiple types: information extraction, multi-session reasoning, knowledge updates, temporal reasoning, and abstention. Overall, the dataset is characterized by extremely long

在第二个注意力图中，峰值出现在位置3、8、12和14，而真实边界位于句子7–8、11–12和13–14之间。总体而言，我们的方法与大多数真实边界实现了紧密对齐，同时提供了更细粒度的分割。这些例子表明，我们的分割方法能够实现细粒度和可靠的主题边界检测，从而验证了其有效性。

C.2 类别精度

如表7所示，检索增强型与内存中心型方法（例如 *A-MEM*、*Mem0*、*MemoryOS*）通常在需要信息整合或信念修正的类别上表现优于 全文，例如 时间、多会话和 知识更新。相比之下，单用户和 单助手等类别中，轻量级检索如 朴素RAG通常具有竞争力，并且可能是最可靠的选择，而 单偏好由于样本量较小而表现出更高的方差。

表7: **类别精度。**按问题类型划分的方法精度（百分比）。括号内表示类别比例和样本量。对于GPT，LightMem配置了参数 $r = 0.7$ 和 $th = 512$ ；对于Qwen，LightMem配置了 $r = 0.4$ 和 $th = 768$ 。

方法	时间 ($n=133$)	多会话 ($n=133$)	知识更新 ($n=78$)	单用户 ($n=70$)	单助手 ($n=56$)	单偏好 ($n=30$)
 GPT-4o-mini						
全文	31.58	45.45	76.92	87.14	89.29	36.67
朴素RAG	39.85	48.48	67.95	90.00	98.21	53.33
<i>LangMem</i>	15.79	20.30	66.67	60.00	46.43	60.00
<i>A-MEM</i>	47.36	48.87	64.11	92.86	96.43	46.67
<i>MemoryOS</i>	32.33	31.06	48.72	80.00	64.29	30.00
<i>Mem0</i>	40.15	46.21	70.12	81.43	41.07	60.00
<i>LightMem</i>	67.18	71.74	83.12	87.14	32.14	68.18
 Qwen3-30B-A3B-Instruct-2507						
全文	33.08	35.61	76.92	82.86	87.50	50.00
朴素RAG	36.84	47.73	65.38	91.43	98.21	70.00
<i>LangMem</i>	37.60	38.35	67.95	78.57	42.86	70.00
<i>A-MEM</i>	51.88	51.12	76.93	90.00	96.43	40.00
<i>MemoryOS</i>	28.57	36.84	61.54	72.86	92.86	33.33
<i>Mem0</i>	41.94	28.13	28.57	55.32	26.09	81.82
<i>LightMem</i>	54.20	51.91	66.67	80.00	31.25	80.00

C.3 详细参数分析

As 表9 显示，我们报告了 LightMem 参数（压缩比率 r 和 STM 阈值 th ）。

D 实验细节

D.1 数据集和基线

数据集 The LongMemEval数据集 (Wu等人,2025) 是用于评估对话代理长期交互记忆的基准数据集。它包含基于扩展的用户-助手对话构建的500个评估问题。它有两个不同长度的版本：LONGMEMEVAL-S设置每个问题包含约115k个标记，而LONGMEMEVAL-M设置在500个会话中扩展到150万个标记。在我们的工作中，我们采用LONGMEMEVAL-S版本，因为它在对话长度和计算可行性之间取得了平衡。问题被分为多种类型：信息提取、多会话推理、知识更新、时间推理和回避。总体而言，该数据集的特点是极长的历史记录、宽泛的时间跨度以及多样的问题类型，使其成为评估对话代理记忆能力的综合基准。在实验过程中，该数据集中有五个样本包含损坏的字符，这导致LightMem的压缩模型无法正常运行。因此，在处理数据集时，LightMem直接丢弃了这五个样本。然而，它们的准确率结果被统一视为错误。这五个样本在数据集索引是74、183、278、351和380。

Table 8: The impact of **LightMem** compression ratio r and STM buffer threshold th is reported here. Due to space limitations, we only present a subset of representative results of the online soft update results, with more results provided in the Figure 9.

Model	th	r	ACC	Input (k)	Output (k)	Total (k)	Calls	Time
	256	0.5	64.29	20.80	10.01	30.81	25.67	302.69
	256	0.6	67.68	24.58	10.53	35.11	30.47	329.61
	256	0.7	65.68	27.66	9.97	37.63	34.26	403.59
	512	0.6	63.74	16.23	9.45	25.68	15.63	266.98
	512	0.7	68.64	18.88	9.37	28.25	18.43	283.76
	512	0.8	66.67	21.55	8.59	30.14	21.11	268.97
	1024	0.6	59.68	10.34	7.68	18.20	7.69	177.45
	1024	0.7	64.68	12.93	6.90	19.83	8.25	209.12
	1024	0.8	64.35	14.86	6.28	21.14	9.43	216.08
	512	0.4	58.57	11.03	17.00	28.03	10.11	421.74
	512	0.6	66.57	16.22	19.50	35.72	15.40	471.09
	512	0.8	67.37	21.35	19.36	40.71	20.98	461.02
	768	0.4	61.95	9.01	16.14	25.15	6.54	357.13
	768	0.6	73.20	13.19	19.21	32.40	9.97	417.13
	768	0.8	64.95	16.94	19.06	36.00	13.09	420.14
	1024	0.4	53.91	8.02	15.44	23.46	4.83	300.56
	1024	0.6	65.67	11.50	18.21	29.71	7.18	396.35
	1024	0.8	68.69	14.82	18.49	33.31	9.43	355.71

histories, wide temporal spans, and diverse question types, making it a comprehensive benchmark for evaluating conversational agents’ memory capabilities. During the experiments, five samples from this dataset contained corrupted characters, which caused LightMem’s compression model to fail to run properly. Consequently, LightMem directly discarded these five samples when processing the dataset. However, their accuracy results were uniformly treated as false. The indices of these five samples in the dataset are 74, 183, 278, 351, and 380.

The LoCoMo benchmark targets the evaluation of long-range conversational memory. It features extremely long dialogues, with each conversation spanning roughly 300 turns and around 9K tokens on average. The accompanying questions fall into four categories—Single-hop, Multi-hop, Temporal, and Open-domain—providing a comprehensive assessment of different dimensions of memory in LLMs.

Baselines We compare our approach against several representative baselines of conversational memory modeling. ① LANGMEM (LangChain, 2025): The Langchain’s long-term memory module. ② A-MEM (Xu et al., 2025): Constructs a memory-centric knowledge graph, encoding each interaction as a structured memory note and linking these notes via LLM-driven reasoning. ③ MEMORYOS (Kang et al., 2025): Organizes conversational memory in an OS-inspired hierarchy, structuring interactions into short-term, mid-term, and long-term layers via paging and heat-based updating. ④ MEM0 (Chhikara et al., 2025): Extracts memories from dialogue turns through a combination of global summaries and recent context, maintaining them via LLM-guided operations.

D.2 IMPLEMENTATION DETAILS

All the experiments are conducted on hardware equipped with 4×NVIDIA RTX 3090 GPUs, dual Intel Xeon Gold 6133 CPUs (40 cores, 80 threads), and 256 GB of RAM.

表8: 展示了**LightMem** 压缩率 r 和STM缓冲区阈值 th 的影响。由于篇幅限制，我们仅展示在线软更新结果的代表性子集，更多结果请参见图9。

模型	th	r	ACC	输入 (k)	输出 (k)	总计 (k)	调用次数	Time
	256	0.5	64.29	20.80	10.01	30.81	25.67	302.69
	256	0.6	67.68	24.58	10.53	35.11	30.47	329.61
	256	0.7	65.68	27.66	9.97	37.63	34.26	403.59
	512	0.6	63.74	16.23	9.45	25.68	15.63	266.98
	512	0.7	68.64	18.88	9.37	28.25	18.43	283.76
	512	0.8	66.67	21.55	8.59	30.14	21.11	268.97
	1024	0.6	59.68	10.34	7.68	18.20	7.69	177.45
	1024	0.7	64.68	12.93	6.90	19.83	8.25	209.12
	1024	0.8	64.35	14.86	6.28	21.14	9.43	216.08
	512	0.4	58.57	11.03	17.00	28.03	10.11	421.74
	512	0.6	66.57	16.22	19.50	35.72	15.40	471.09
	512	0.8	67.37	21.35	19.36	40.71	20.98	461.02
	768	0.4	61.95	9.01	16.14	25.15	6.54	357.13
	768	0.6	73.20	13.19	19.21	32.40	9.97	417.13
	768	0.8	64.95	16.94	19.06	36.00	13.09	420.14
	1024	0.4	53.91	8.02	15.44	23.46	4.83	300.56
	1024	0.6	65.67	11.50	18.21	29.71	7.18	396.35
	1024	0.8	68.69	14.82	18.49	33.31	9.43	355.71

历史记录、宽泛的时间跨度以及多样的问题类型，使其成为评估对话代理记忆能力的综合基准。在实验过程中，该数据集中有五个样本包含损坏的字符，这导致LightMem的压缩模型无法正常运行。因此，在处理数据集时，LightMem直接丢弃了这五个样本。然而，它们的准确率结果被统一视为错误。这些五个样本在数据集中的索引是74、183、278、351和380。

LoCoMo基准测试旨在评估长程对话记忆能力。它包含极长的对话，每个对话大约包含300轮，平均约9K个token。配套的问题分为四个类别——单跳、多跳、时序和开放域——全面评估了大语言模型中记忆的不同维度。

基线 我们将我们的方法与几种具有代表性的对话记忆建模基线进行了比较。1 LANGMEM (LangChain,2025): Langchain的长期记忆模块。2 A-MEM (Xu 等人, 2025): 构建以记忆为中心的知识图谱，将每次交互编码为结构化的记忆笔记，并通过大语言模型驱动的推理将这些笔记连接起来。3 MEM-ORYOS (Kang 等人 ,2025): 受操作系统启发，组织对话记忆的层次结构，通过分页和基于热度的更新将交互结构化为短期、中期和长期层。4 MEM0 (Chhikara 等人, 2025): 通过结合全局摘要和最近上下文从对话轮中提取记忆，并通过大语言模型指导的操作进行维护。

D.2 实现细节

所有实验均在配备 4×NVIDIA RTX 3090 GPU、双Intel Xeon Gold 6133 CPU（40核，80线程）和256 GB RAM的硬件上运行。

Table 9: The impact of LightMem’s compression ratio (r) and STM buffer threshold (th).

Model	th	r	ACC	Input (k)	Output (k)	Total (k)	Calls	Time
 GPT-4o-mini	0	0.4	58.04	27.70	8.90	36.60	39.91	500.69
	256	0.4	57.78	16.64	8.40	25.04	20.25	254.93
	512	0.4	55.56	11.05	7.66	18.71	10.13	230.59
	768	0.4	49.29	9.05	6.55	15.60	6.57	157.13
	1024	0.4	46.87	7.75	5.25	13.00	4.82	118.11
	0	0.5	62.89	30.84	9.75	40.59	43.56	550.36
	256	0.5	64.29	20.80	10.01	30.81	25.67	302.69
	512	0.5	62.44	13.49	8.89	22.38	12.70	250.36
	768	0.5	56.12	10.93	7.57	18.50	8.12	203.13
	1024	0.5	50.36	8.34	6.97	15.31	6.32	160.35
	0	0.6	70.35	33.17	10.20	43.37	45.86	553.07
	256	0.6	67.68	24.58	10.53	35.11	30.47	329.61
	512	0.6	63.74	16.23	9.45	25.68	15.63	266.98
	768	0.6	64.44	13.04	8.10	21.14	9.90	210.05
	1024	0.6	59.68	10.34	7.68	18.20	7.69	177.45
	0	0.7	62.35	35.36	9.76	45.12	48.08	573.42
	256	0.7	65.68	27.66	9.97	37.63	34.26	403.59
	512	0.7	68.64	18.88	9.37	28.25	18.43	283.76
	1024	0.7	64.68	12.93	6.90	19.83	8.25	209.12
	0	0.8	61.52	39.32	9.89	49.21	52.97	622.90
	256	0.8	66.37	30.67	9.70	40.37	41.66	489.61
	512	0.8	66.67	21.55	8.59	30.14	21.11	268.97
	1024	0.8	64.35	14.86	6.28	21.14	9.43	216.08
 Qwen3	0	0.4	56.89	28.44	18.30	46.74	41.08	594.94
	256	0.4	52.37	16.82	17.63	34.45	20.48	450.98
	512	0.4	58.57	11.03	17.00	28.03	10.11	421.74
	768	0.4	61.95	9.01	16.14	25.15	6.54	357.13
	1024	0.4	53.91	8.02	15.44	23.46	4.83	300.56
	0	0.6	69.56	34.90	20.26	55.16	48.63	642.10
	256	0.6	65.37	24.78	19.59	44.37	30.66	520.37
	512	0.6	66.57	16.22	19.50	35.72	15.40	471.09
	768	0.6	73.20	13.19	19.21	32.40	9.97	417.13
	1024	0.6	65.67	11.50	18.21	29.71	7.18	396.35
	0	0.8	67.68	37.97	20.18	58.15	50.81	759.15
	256	0.8	64.52	30.54	19.77	50.31	37.35	550.98
	512	0.8	67.37	21.35	19.36	40.71	20.98	461.02
	768	0.8	64.95	16.94	19.06	36.00	13.09	420.14
	1024	0.8	68.69	14.82	18.49	33.31	9.43	355.71

表9: LightMem的压缩率(r)和STM缓冲区阈值(th)的影响。

模型	th	r	ACC	输入 (千字节)	输出 (千字节)	总计 (千字节)	调用次数	Time
 GPT-4o-mini	0	0.4	58.04	27.70	8.90	36.60	39.91	500.69
	256	0.4	57.78	16.64	8.40	25.04	20.25	254.93
	512	0.4	55.56	11.05	7.66	18.71	10.13	230.59
	768	0.4	49.29	9.05	6.55	15.60	6.57	157.13
	1024	0.4	46.87	7.75	5.25	13.00	4.82	118.11
	0	0.5	62.89	30.84	9.75	40.59	43.56	550.36
	256	0.5	64.29	20.80	10.01	30.81	25.67	302.69
	512	0.5	62.44	13.49	8.89	22.38	12.70	250.36
	768	0.5	56.12	10.93	7.57	18.50	8.12	203.13
	1024	0.5	50.36	8.34	6.97	15.31	6.32	160.35
	0	0.6	70.35	33.17	10.20	43.37	45.86	553.07
	256	0.6	67.68	24.58	10.53	35.11	30.47	329.61
	512	0.6	63.74	16.23	9.45	25.68	15.63	266.98
	768	0.6	64.44	13.04	8.10	21.14	9.90	210.05
	1024	0.6	59.68	10.34	7.68	18.20	7.69	177.45
	0	0.7	62.35	35.36	9.76	45.12	48.08	573.42
	256	0.7	65.68	27.66	9.97	37.63	34.26	403.59
	512	0.7	68.64	18.88	9.37	28.25	18.43	283.76
	1024	0.7	64.68	12.93	6.90	19.83	8.25	209.12
	0	0.8	61.52	39.32	9.89	49.21	52.97	622.90
	256	0.8	66.37	30.67	9.70	40.37	41.66	489.61
	512	0.8	66.67	21.55	8.59	30.14	21.11	268.97
	1024	0.8	64.35	14.86	6.28	21.14	9.43	216.08
 Qwen3	0	0.4	56.89	28.44	18.30	46.74	41.08	594.94
	256	0.4	52.37	16.82	17.63	34.45	20.48	450.98
	512	0.4	58.57	11.03	17.00	28.03	10.11	421.74
	768	0.4	61.95	9.01	16.14	25.15	6.54	357.13
	1024	0.4	53.91	8.02	15.44	23.46	4.83	300.56
	0	0.6	69.56	34.90	20.26	55.16	48.63	642.10
	256	0.6	65.37	24.78	19.59	44.37	30.66	520.37
	512	0.6	66.57	16.22	19.50	35.72	15.40	471.09
	768	0.6	73.20	13.19	19.21	32.40	9.97	417.13
	1024	0.6	65.67	11.50	18.21	29.71	7.18	396.35
	0	0.8	67.68	37.97	20.18	58.15	50.81	759.15
	256	0.8	64.52	30.54	19.77	50.31	37.35	550.98
	512	0.8	67.37	21.35	19.36	40.71	20.98	461.02
	768	0.8	64.95	16.94	19.06	36.00	13.09	420.14
	1024	0.8	68.69	14.82	18.49	33.31	9.43	355.71

E PROMPTS

E.1 LLM-AS-JUDGE

Standard Tasks (Single-session-user/assistant Multi-session)
I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.
Temporal Reasoning Tasks
I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response is equivalent to the correct answer or contains all the intermediate steps to get the correct answer, you should also answer yes. If the response only contains a subset of the information required by the answer, answer no. In addition, do not penalize off-by-one errors for the number of days. If the question asks for the number of days/weeks/months, etc., and the model makes off-by-one errors (e.g., predicting 19 days when the answer is 18), the model’s response is still correct. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.
Knowledge Update Tasks
I will give you a question, a correct answer, and a response from a model. Please answer yes if the response contains the correct answer. Otherwise, answer no. If the response contains some previous information along with an updated answer, the response should be considered as correct as long as the updated answer is the required answer. Question: {question} Correct Answer: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.
Single-session Preference Tasks
I will give you a question, a rubric for desired personalized response, and a response from a model. Please answer yes if the response satisfies the desired response. Otherwise, answer no. The model does not need to reflect all the points in the rubric. The response is correct as long as it recalls and utilizes the user’s personal information correctly. Question: {question} Rubric: {answer} Model Response: {response} Is the model response correct? Answer yes or no only.

E 提示

E.1 LLM-AS-JUDGE

标准任务（单会话用户/助手多会话）
我会给你一个问题和一个正确答案和一个模型的回应。如果回应中包含正确答案，请回答yes。否则，回答no。如果回应与正确答案等效，或者包含得到正确答案的所有中间步骤，你也应该回答yes。如果回应仅包含答案所需信息的一个子集，回答no。 问题: {问题}正确答案: {答案}模型回应: {回应}模型回应是否正确? 仅回答yes或no。
时间推理任务
我将给你一个问题和一个正确答案和一个模型的回应。如果回应中包含正确答案，请回答“是”。否则，请回答“否”。如果回应与正确答案等效，或者包含所有得到正确答案的中间步骤，你也应该回答“是”。如果回应仅包含所需答案信息的一个子集，请回答“否”。此外，对于天数等数值，不要因为个位数误差而扣分。如果问题要求的是天数/周数/月数等，而模型出现了个位数误差（例如预测为19天，而正确答案是18天），模型的回应仍然被认为是正确的。 问题: {问题}正确答案: {答案}模型回答: {回答}模型回答是否正确? 仅回答是或否。
知识更新任务
我将给你一个问题和一个正确答案和一个模型的回应。如果回应中包含正确答案，请回答“是”。否则，请回答“否”。如果回应中包含一些先前信息以及一个更新后的答案，只要更新后的答案是所需的答案，该回应应被视为正确。 问题: {问题}正确答案: {答案}模型响应: {响应}模型响应是否正确? 仅回答是或否。
单会话偏好任务
我会给你一个问题和一个期望个性化响应的评分标准，以及一个模型的响应。如果响应满足期望的响应，请回答是。否则，回答否。模型不需要反映评分标准中的所有要点。只要它正确回忆和利用了用户的个人信息，响应就是正确的。 问题: {问题}评分标准: {答案}模型回答: {回答}模型回答是否正确? 仅回答是或否。

Abstention Tasks

I will give you an unanswerable question, an explanation, and a response from a model. Please answer yes if the model correctly identifies the question as unanswerable. The model could say that the information is incomplete, or some other information is given but the asked information is not.
Question: {question}
Explanation: {answer}
Model Response: {response}
Does the model correctly identify the question as unanswerable? Answer yes or no only.

弃权任务

我会给你一个无解的问题、一个解释以及来自模型的回答。请回答是，如果模型正确识别问题为无解。模型可能会说信息不完整，或者提供了其他信息但未给出所问信息。**问题：** {问题}**解释：** {答案}**模型回答：** {回答}模型是否正确识别问题为无解？仅回答是或否。